KOLBEN ERP

Documentación técnica de infraestructura y desarrollo

Versión 1.1 · 24 de octubre de 2026

Desarrollo: Diego Aguilar · Cliente: Javier Escalante (Kolben)

Documento vivo: se ajusta conforme avance el proyecto. Las funciones del sistema están en la Especificación funcional v1.0 (23/10/2026); este documento cubre cómo se construye, se prueba y se publica.

1. Resumen y entornos

Kolben ERP es un sistema web administrativo interno (ERP a la medida, un solo usuario con control total) para Kolben, distribuidora de repuestos (cilindros de freno y clutch). Reemplaza el control actual en hojas sueltas y cubre el ciclo completo: productos y precios por cliente, proformas y facturas, cuentas por cobrar, inventario por lotes (FIFO), pedidos al proveedor y costeo de importación, utilidad, caja y resumen ejecutivo.

Relación con CatalogoHN

- **Proyectos 100% separados:** CatalogoHN (`catalogohn.com`, SaaS multi-inquilino de catálogos, puerto 3001) y Kolben ERP tienen repositorio, código, modelos y base de datos independientes.
- **Integración futura (opcional):** el ERP podría conectarse por API REST/Webhooks con CatalogoHN (por ejemplo, descontar stock público al facturar o recibir pedidos como proformas). Hoy no existe; los pedidos de CatalogoHN se ingresan a mano en el POS. El backend se diseña con una API limpia bajo `/api` para dejar esa puerta abierta.

Entornos

Ambos proyectos conviven en el mismo Droplet de DigitalOcean (Ubuntu LTS), con Nginx como proxy inverso y certificados gratuitos de Let's Encrypt. Kolben ERP tiene dos entornos completos: **pruebas** (rama `develop`) y **producción** (rama `main`), cada uno con puerto, carpeta, proceso PM2 y base de datos propios, dentro de la misma instancia de PostgreSQL pero aisladas de la base de CatalogoHN.

	Pruebas (test)	Producción
Rama de Git	`develop`	`main`
Dominio	test.app.kolben.catalogohn.com	app.kolben.catalogohn.com
Puerto API (interno)	4001	4002
Base de datos	`kolben_erp_test`	`kolben_erp`
Carpeta en el Droplet	`/var/www/kolben-erp-test`	`/var/www/kolben-erp`
Proceso PM2	`kolben-api-test`	`kolben-api-prod`
Archivos subidos	`/var/www/kolben-erp-test/backend/uploads/`	`/var/www/kolben-erp/backend/uploads/`
Frontend (estático)	`.../frontend/dist` servido por Nginx	`.../frontend/dist` servido por Nginx

Nota: Falta crear en el DNS un registro A por cada subdominio apuntando a la IP del Droplet. Cuando exista un nombre más corto, solo cambia `server_name`, `APP_URL` y el certificado.

Cómo se conectan las piezas

- El navegador entra por HTTPS a Nginx (puertos 80/443). Nginx entrega el frontend compilado (React) y envía todo lo que empiece con `/api/` al backend en `127.0.0.1:4001` o `:4002`.
- Como frontend y API comparten el mismo dominio, la cookie HttpOnly del JWT funciona sin CORS ni configuración especial.

- Los puertos 4001 y 4002 **no se abren al exterior**: el backend escucha solo en `127.0.0.1`. CatalogoHN usa el 3001; no hay conflicto.
- PostgreSQL escucha solo en `localhost`. Pruebas y producción usan bases y usuarios distintos, y ninguno tiene permiso sobre la base de CatalogoHN.

2. Stack tecnológico

Capa	Tecnología	Notas
Frontend	React + Vite + TypeScript	SPA con ancho fijo de 1280 px (según la spec).
UI	Tailwind CSS + shadcn/ui	Tipografía Inter con números tabulares.
Tablas	`@tanstack/react-table`	Ventas, Productos, Clientes, flujo de Caja.
Backend	Node.js + Express + TypeScript	API REST bajo `/api`.
Base de datos	PostgreSQL local en el Droplet	Una base por entorno.
ORM	Prisma	Migraciones versionadas en el repositorio.
Autenticación	JWT en cookie HttpOnly	Contraseña con hash (bcrypt o argon2). Cookie `Secure` + `SameSite=Lax`.
Archivos	Disco local del Droplet	Nginx sirve vía endpoint protegido (sección 7).
PDFs	`@react-pdf/renderer`	Proforma, factura, catálogo, cotización, respaldo de Por cobrar. Se generan en el backend con `renderToBuffer` para poder enviarlos por correo.
Moneda	`decimal.js` + `Intl.NumberFormat("es-HN")`	Confirmado. Columnas `NUMERIC` en PostgreSQL; nunca `float`. Lempiras con 2 decimales en pantalla.
Excel	`exceljs`	Confirmado. La spec lo pide: cotización de la Calculadora y descargable de pedido.
Imágenes	`sharp`	Redimensiona fotos y logos al subirlas: menos peso en disco y catálogo PDF más rápido.
Validación	`zod`	Mismo esquema para validar entradas del API.
Procesos programados	`node-cron`	Cierre de mes, respaldo semanal por correo. PM2 en modo fork con 1 instancia.
Correo	`nodemailer`	Respaldo semanal de Por cobrar (PDF).
Seguridad	`helmet`, `express-rate-limit`	Límite de intentos en el login.
Procesos / Web	PM2 + Nginx + Certbot	HTTPS con Let's Encrypt.

3. Estructura del repositorio (monorepo)

Un solo repositorio de GitHub con dos aplicaciones. Cada entorno del Droplet es un clon del mismo repositorio, parado en su rama.

```
kolben-erp/
├── backend/
│   ├── prisma/
│   │   ├── schema.prisma
│   │   ├── seed.ts              # datos de prueba (solo local y pruebas)
```

```
│   │   └── migrations/
│   ├── src/
│   │   ├── server.ts            # arranque (escucha en 127.0.0.1)
│   │   ├── app.ts               # express, helmet, rutas
│   │   ├── config/env.ts        # lee y valida .env con zod
│   │   ├── middlewares/         # auth (JWT cookie), errores, rate limit
│   │   ├── modules/
│   │   │   ├── auth/
│   │   │   ├── productos/       # productos, categorias, marcas, lotes, ajustes
│   │   │   ├── clientes/        # clientes, zonas, ciudades, precios
│   │   │   ├── pos/             # proformas
│   │   │   ├── ventas/          # facturas, abonos, cambios
│   │   │   ├── compras/         # calculadora, pedidos, pagos, recepcion
│   │   │   ├── caja/            # saldos, gastos, flujo, cierre de anio
│   │   │   ├── reportes/        # resumen, ventas/utilidad por codigo
│   │   │   └── catalogo/
│   │   ├── services/            # fifo, costeo, multiplicador, pe-meta
│   │   ├── pdf/                 # plantillas react-pdf
│   │   ├── jobs/                # cierre-mes, respaldo-semanal
│   │   └── lib/                 # prisma client, decimal, historial
│   ├── uploads/                 # NO se sube a Git
│   │   ├── facturas/  logos/  comprobantes/  productos/  catalogo/
│   ├── .env                     # NO se sube a Git
│   ├── package.json
│   └── tsconfig.json
├── frontend/
│   ├── src/
│   │   ├── pages/               # Resumen, POS, Ventas, Clientes, ...
│   │   ├── components/ui/       # shadcn/ui
│   │   ├── components/          # tablas, formularios, layout
│   │   ├── lib/                 # api (fetch), formato de moneda y fechas
│   │   └── main.tsx
│   ├── .env.production          # VITE_API_URL=/api
│   ├── vite.config.ts
│   └── package.json
├── scripts/
│   ├── deploy.sh
│   ├── backup.sh
│   ├── caja_backup.sh
│   └── reset-password.ts    # restablece la contraseña (se corre en el servidor)
├── ecosystem.config.js          # PM2
├── docs/                        # especificacion y este documento
├── .gitignore
└── README.md
```

.gitignore mínimo: `node_modules/`, `dist/`, `.env`, `.env.*` (excepto los `.example`), `backend/uploads/*` y `*.dump`.

4. Variables de entorno

Cada entorno tiene su propio `backend/.env` en el servidor (nunca en Git). En el repositorio solo va `backend/.env.example` con valores de ejemplo.

backend/.env (producción)

```
NODE_ENV=production
HOST=127.0.0.1
PORT=4002
TZ=America/Tegucigalpa

DATABASE_URL="postgresql://kolben_prod:CAMBIAR_PASSWORD@localhost:5432/kolben_erp?schema=pub
lic"

JWT_SECRET=CAMBIAR_por_cadena_larga_aleatoria_64_caracteres
```

```
JWT_EXPIRES_IN=8h
COOKIE_NAME=kolben_token
COOKIE_SECURE=true

APP_URL=https://app.kolben.catalogohn.com
UPLOADS_DIR=/var/www/kolben-erp/backend/uploads

SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=correo@ejemplo.com
SMTP_PASS=CAMBIAR_app_password
SMTP_FROM="Kolben ERP <correo@ejemplo.com>"
```

Qué cambia en pruebas

Variable	Pruebas	Local (tu computadora)
NODE_ENV	`test`	`development`
PORT	`4001`	`4000`
DATABASE_URL	`.../kolben_erp_test` con usuario `kolben_test`	`.../kolben_erp_local`
COOKIE_SECURE	`true`	`false` (sin HTTPS)
APP_URL	https://test.app.kolben.catalogohn.com	http://localhost:5173
UPLOADS_DIR	`/var/www/kolben-erp-test/backend/uploads`	`./uploads`
JWT_SECRET	Distinto al de producción	Cualquiera

En pruebas conviene apuntar el correo a tu propio correo para no enviar el respaldo semanal a Javier por error.

frontend/.env.production

```
VITE_API_URL=/api
```

Al ser ruta relativa, el mismo build funciona en pruebas y producción. En local, `vite.config.ts` define un proxy de `/api` hacia `http://localhost:4000`.

5. Flujo de trabajo con Git

Todo cambio se desarrolla en tu computadora, se sube a `develop`, se prueba en el entorno de pruebas del Droplet y, solo si está bien, se pasa a `main` y a producción.

```
tu PC  ──push──▶  GitHub: develop  ──git pull──▶  Droplet: PRUEBAS
                        │ (si todo está bien)
                      merge
                        ▼
                  GitHub: main  ──git pull──▶  Droplet: PRODUCCIÓN
```

Paso 1 · Preparar el repositorio (una sola vez)

```
git init
git remote add origin git@github.com:TU_USUARIO/kolben-erp.git
git checkout -b main
git add .
git commit -m "chore: estructura inicial"
```

```
git push -u origin main
git checkout -b develop
git push -u origin develop
```

Recomendado: en GitHub, proteger `main` (Settings → Branches) para no subir directo por error.

Paso 2 · Trabajar y subir a develop (día a día)

```
git checkout develop
git pull origin develop
# ... programar ...
git add .
git commit -m "feat(pos): guardar proforma y apartar unidades"
git push origin develop
```

Prefijos de commit sugeridos: `feat`, `fix`, `chore`, `docs`, `refactor`. Si un cambio es grande, crea una rama `feature/nombre` desde `develop`, y luego haz merge a `develop`.

Paso 3 · Desplegar en PRUEBAS (Droplet)

```
ssh usuario@IP_DEL_DROPLET
cd /var/www/kolben-erp-test
git pull origin develop

cd backend
npm ci
npx prisma migrate deploy
npx prisma generate
npm run build

cd ../frontend
npm ci
npm run build

pm2 restart kolben-api-test
pm2 logs kolben-api-test --lines 50
```

Prueba en https://test.app.kolben.catalogohn.com. Si algo falla, corriges en tu PC y repites desde el Paso 2.

Paso 4 · Pasar a main (cuando pruebas está bien)

```
git checkout main
git pull origin main
git merge develop
git push origin main
git tag -a v1.0.0 -m "Primera versión estable"
git push origin v1.0.0
git checkout develop
```

Paso 5 · Desplegar en PRODUCCIÓN (Droplet)

Antes de tocar producción, haz siempre un respaldo (el script está en la sección 8).

```
ssh usuario@IP_DEL_DROPLET
/var/www/kolben-erp/scripts/backup.sh

cd /var/www/kolben-erp
git pull origin main

cd backend
npm ci
npx prisma migrate deploy
npx prisma generate
npm run build
```

```
cd ../frontend
npm ci
npm run build

pm2 restart kolben-api-prod
pm2 logs kolben-api-prod --lines 50
```

Script opcional: deploy.sh

Hace los mismos pasos con un solo comando (`./scripts/deploy.sh test` o `./scripts/deploy.sh prod`). Sigue siendo manual: lo ejecutas tú.

```
#!/bin/bash
set -e
ENV=$1
if [ "$ENV" = "test" ]; then
  DIR=/var/www/kolben-erp-test; BRANCH=develop; APP=kolben-api-test
elif [ "$ENV" = "prod" ]; then
  DIR=/var/www/kolben-erp; BRANCH=main; APP=kolben-api-prod
  /var/www/kolben-erp/scripts/backup.sh
else
  echo "Uso: ./scripts/deploy.sh test|prod"; exit 1
fi

cd $DIR
git pull origin $BRANCH
cd backend && npm ci && npx prisma migrate deploy && npx prisma generate
npm run build
cd ../frontend && npm ci && npm run build
pm2 restart $APP
echo "Listo: $APP"
```

Migraciones de base de datos

- En tu PC: `npx prisma migrate dev --name descripcion_del_cambio` crea y aplica la migración. La carpeta `prisma/migrations` **sí se sube a Git**.
- En el Droplet: solo `npx prisma migrate deploy` (aplica las pendientes, nunca borra datos).
- Nunca uses `migrate dev` ni `migrate reset` en producción.

Volver a una versión anterior

```
cd /var/www/kolben-erp
git fetch --tags
git checkout v1.0.0
# repetir build de backend y frontend, luego:
pm2 restart kolben-api-prod
# volver a seguir main después:
git checkout main
```

Si la versión anterior tenía otra estructura de base de datos, restaura primero el respaldo (sección 8).

6. Preparación del Droplet (una sola vez)

Paso 1 · Paquetes base

```
sudo apt update && sudo apt install -y nginx postgresql git certbot python3-certbot-nginx
rclone
# Node.js 20 LTS (o 22 LTS); usar la misma versión que ya corre CatalogoHN
```

```
sudo npm install -g pm2
```

Paso 2 · Bases de datos y usuarios separados

```
sudo -u postgres psql
CREATE USER kolben_prod WITH PASSWORD 'CAMBIAR_1';
CREATE USER kolben_test WITH PASSWORD 'CAMBIAR_2';
CREATE DATABASE kolben_erp OWNER kolben_prod;
CREATE DATABASE kolben_erp_test OWNER kolben_test;
\q
```

Paso 3 · Clonar el repositorio dos veces

```
cd /var/www
git clone -b main    git@github.com:TU_USUARIO/kolben-erp.git kolben-erp
git clone -b develop git@github.com:TU_USUARIO/kolben-erp.git kolben-erp-test

mkdir -p kolben-erp/backend/uploads/{facturas,logos,comprobantes,productos,catalogo}
mkdir -p kolben-erp-test/backend/uploads/{facturas,logos,comprobantes,productos,catalogo}
# crear backend/.env en cada carpeta (sección 4), luego primer despliegue
```

Paso 4 · Firewall

```
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw enable
# NO abrir 4001 ni 4002
```

Si el Droplet ya tiene reglas de firewall por CatalogoHN, solo verifica que 4001 y 4002 no estén abiertos.

Paso 5 · PM2 (ecosystem.config.js en la raíz del repo)

```
module.exports = {
  apps: [
    {
      name: 'kolben-api-test',
      cwd: '/var/www/kolben-erp-test/backend',
      script: 'dist/server.js',
      instances: 1,
      exec_mode: 'fork',
      max_memory_restart: '512M',
    },
    {
      name: 'kolben-api-prod',
      cwd: '/var/www/kolben-erp/backend',
      script: 'dist/server.js',
      instances: 1,
      exec_mode: 'fork',
      max_memory_restart: '512M',
    },
  ],
};
```

```
pm2 start /var/www/kolben-erp/ecosystem.config.js
pm2 save
pm2 startup        # ejecutar el comando que muestre
pm2 status
```

Cada proceso lee su propio `.env` (PORT, base de datos, etc.) desde su carpeta. El límite de 512 MB por proceso deja margen para generar PDFs con fotos (catálogo); con el Droplet en torno al 45% de RAM hay espacio, pero conviene vigilarlo con `pm2 monit` la primera vez que se genere el catálogo.

Debe ser **una sola instancia**, porque los procesos programados (cierre de mes, respaldo semanal) no deben ejecutarse duplicados.

Paso 6 · Nginx y HTTPS

Archivo `/etc/nginx/sites-available/kolben-erp` (producción):

```
server {
    listen 80;
    server_name app.kolben.catalogohn.com;

    root /var/www/kolben-erp/frontend/dist;
    index index.html;
    client_max_body_size 10M;

    location / {
        try_files $uri /index.html;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:4002;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Archivos privados: solo se entregan si el backend lo autoriza
    location /protected-uploads/ {
        internal;
        alias /var/www/kolben-erp/backend/uploads/;
    }
}
```

Archivo `/etc/nginx/sites-available/kolben-erp-test` (pruebas): igual, cambiando `server_name test.app.kolben.catalogohn.com`, el `root` a `/var/www/kolben-erp-test/frontend/dist`, el puerto a `4001` y el `alias` a `/var/www/kolben-erp-test/backend/uploads/`. Se recomienda agregar `add_header X-Robots-Tag "noindex";`.

```
sudo ln -s /etc/nginx/sites-available/kolben-erp /etc/nginx/sites-enabled/
sudo ln -s /etc/nginx/sites-available/kolben-erp-test /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d app.kolben.catalogohn.com -d test.app.kolben.catalogohn.com
```

Certbot agrega el HTTPS y la redirección automáticamente, y renueva el certificado solo.

7. Archivos subidos (facturas, logos, comprobantes)

Todo se guarda en `backend/uploads/` del entorno correspondiente, ordenado por tipo: `facturas/`, `logos/` (marcas), `comprobantes/`, `productos/` (fotos) y `catalogo/` (portada).

- **Recomendado: endpoint protegido.** El navegador pide `/api/files/:tipo/:nombre`; el backend valida la cookie JWT y responde con la cabecera `X-Accel-Redirect: /protected-uploads/tipo/nombre`. Nginx entrega el archivo directamente, sin pasar el contenido por Node. Sin sesión, no hay acceso.
- Alternativa (solo para archivos no sensibles como logos o fotos de productos): `location /uploads/` estático. No recomendado para facturas ni comprobantes.
- En la base de datos se guarda solo la ruta relativa (por ejemplo `facturas/1504.pdf`), nunca la ruta absoluta.

- Subidas con `multer`: validar tipo (PDF, JPG, PNG) y tamaño máximo; renombrar el archivo con un identificador propio.
- La carpeta `uploads/` entra en el respaldo diario (sección 8) y **no** se versiona en Git.

8. Respaldos automáticos

La spec exige una copia diaria de la base de datos **fuera del servidor**. Se propone lo siguiente para producción (en pruebas no es necesario):

Qué	Cuándo	Retención	Contenido
Base de datos completa	Diario 2:00 a. m.	30 días en el Droplet; copia externa	`pg_dump` formato custom de `kolben_erp`
Carpeta `uploads/`	Diario 2:00 a. m.	30 días	Archivo `.tar.gz`
Datos de Caja (CSV)	Semanal, lunes 2:30 a. m.	12 semanas	Saldos, bancos, gastos fijos y variables
PDF de Por cobrar	Semanal, día configurado	En el correo	Lo envía el sistema (spec, sección 7)

Configurar rclone con Google Drive (una sola vez)

El Droplet no tiene navegador, así que la autorización de Google se hace desde tu computadora.

```
# 1) En el Droplet
sudo apt install -y rclone
rclone config
#   n) New remote  → nombre: kolben-remoto → tipo: drive
#   scope: drive.file (solo ve lo que rclone crea)
#   "Use web browser to automatically authenticate?" → n
#   copia el comando "rclone authorize" que muestra

# 2) En tu computadora (con rclone instalado)
rclone authorize "drive" "TOKEN_QUE_MOSTRO_EL_DROPLET"
#   se abre el navegador, inicia sesión y acepta; copia el token que imprime

# 3) De vuelta en el Droplet: pega el token y termina la configuración
rclone lsd kolben-remoto:     # prueba de conexión
rclone mkdir kolben-remoto:kolben-backups
```

El token queda guardado en `~/.config/rclone/rclone.conf` del usuario que corre el cron: protégelo (`chmod 600`) y no lo subas a Git. Usar una cuenta de Google propia del negocio, no una personal compartida.

Credenciales sin contraseña en el script

```
# archivo ~/.pgpass del usuario que corre el cron (permisos 600)
localhost:5432:kolben_erp:kolben_prod:CAMBIAR_1
chmod 600 ~/.pgpass
```

scripts/backup.sh

```
#!/bin/bash
set -e
# si cualquier paso falla, avisa a Healthchecks.io (que envía correo)
trap 'curl -fsS -m 10 https://hc-ping.com/TU_UUID_BACKUP/fail > /dev/null' ERR
FECHA=$(date +%Y-%m-%d_%H%M)
DEST=/var/backups/kolben
```

```
mkdir -p $DEST

pg_dump -U kolben_prod -h localhost -Fc kolben_erp > $DEST/kolben_erp_$FECHA.dump
tar -czf $DEST/uploads_$FECHA.tar.gz -C /var/www/kolben-erp/backend uploads

# borrar copias locales de más de 30 días
find $DEST -type f -mtime +30 -delete

# copia fuera del servidor (Google Drive, remoto configurado como 'kolben-remoto')
rclone copy $DEST kolben-remoto:kolben-backups --max-age 2d
# aviso de éxito: si el ping no llega a tiempo, Healthchecks.io también avisa
curl -fsS -m 10 --retry 3 https://hc-ping.com/TU_UUID_BACKUP > /dev/null
echo "Respaldo OK $FECHA"
```

scripts/caja_backup.sh

```
#!/bin/bash
set -e
FECHA=$(date +%Y-%m-%d)
DEST=/var/backups/kolben/caja
mkdir -p $DEST
for T in SaldoCaja Banco GastoFijoMes MovimientoGasto CategoriaGasto Anio MesContable; do
  psql -U kolben_prod -h localhost -d kolben_erp \
    -c "\copy (SELECT * FROM \"$T\") TO '$DEST/$T-$FECHA.csv' CSV HEADER"
done
find $DEST -type f -mtime +84 -delete
rclone copy $DEST kolben-remoto:kolben-backups/caja --max-age 8d
```

Cron (sudo crontab -e)

```
0 2 * * *   /var/www/kolben-erp/scripts/backup.sh      >> /var/log/kolben-backup.log 2>&1
30 2 * * 1  /var/www/kolben-erp/scripts/caja_backup.sh >> /var/log/kolben-backup.log 2>&1
```

Avisos y mantenimiento

- **Aviso si falla el respaldo:** cuenta gratuita en Healthchecks.io. El script le hace un ping al terminar bien y otro a `/fail` si algo sale mal; si el ping no llega a la hora esperada, Healthchecks envía un correo. Usar un chequeo distinto para `caja_backup.sh`.
- **Rotación de logs:** que `/var/log/kolben-backup.log` y los logs de PM2 no crezcan sin límite (`pm2 install pm2-logrotate`; para el log de respaldo, un archivo en `/etc/logrotate.d/`).
- **Espacio en disco:** revisar de vez en cuando con `df -h` el crecimiento de `uploads/` y `/var/backups/kolben`.

Restaurar un respaldo

```
pg_restore -U kolben_prod -h localhost -d kolben_erp --clean --if-exists \
  /var/backups/kolben/kolben_erp_AAAA-MM-DD_HHMM.dump
tar -xzf uploads_AAAA-MM-DD_HHMM.tar.gz -C /var/www/kolben-erp/backend
```

Probar la restauración en la base de pruebas al menos una vez antes del arranque. Un respaldo que nunca se ha restaurado no está comprobado.

9. Modelo de datos inicial (Prisma)

El diagrama de relaciones está en el **Anexo B**. Basado en la sección 2 de la especificación. Es el punto de partida (`backend/prisma/schema.prisma`); se ajustará con las migraciones. Los montos usan `Decimal`; los cálculos internos van con más decimales que los 2 que se muestran.

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum TipoVenta {
  CREDITO
  CONTADO
}
enum EstadoProforma {
  PENDIENTE
  INGRESADA
  ELIMINADA
}
enum TipoLinea {
  NORMAL
  CAMBIO
}
enum EstadoPedido {
  EN_PRODUCCION
  EN_CAMINO
  RECIBIDO
}
enum TipoPagoPedido {
  PROVEEDOR
  GASTOS
}
enum MedioAbono {
  EFECTIVO
  TRANSFERENCIA
  CHEQUE
  DEVOLUCION
}
enum GrupoGasto {
  NEGOCIO
  PERSONAL
}
enum TipoGasto {
  FIJO
  VARIABLE
}

// ---------- Acceso y configuración ----------
model Usuario {
  id           Int      @id @default(autoincrement())
  usuario      String   @unique
  passwordHash String
  creadoEn     DateTime @default(now())
}

model Configuracion {            // registro único (id = 1)
  id             Int     @id @default(1)
  vendedor       String
  contacto       String
  titularCheques String?
  porcentajeIsv  Decimal @default(15) @db.Decimal(5, 2)
  correoRespaldo String?
```

```
  diaRespaldo    Int     @default(1)
  pctMesSinStock Decimal @default(100) @db.Decimal(5, 2) // % de días del mes en cero para
contar como mes sin stock (100 = todo el mes)
}

model Banco {
  id                Int     @id @default(autoincrement())
  nombre            String
  cuentaEnmascarada String
  orden             Int
  saldoLempiras     Decimal @default(0) @db.Decimal(14, 2)
}

// ---------- Listas ----------
model Categoria {
  id        Int        @id @default(autoincrement())
  nombre    String     @unique
  orden     Int
  empaque   Int        @default(10)     // 10 / 20 / 50
  productos Producto[]
}

model Marca {
  id        Int        @id @default(autoincrement())
  nombre    String     @unique
  logoPath  String?
  orden     Int
  productos Producto[]
}

model Zona {
  id       Int      @id @default(autoincrement())
  nombre   String   @unique
  orden    Int
  ciudades Ciudad[]
}

model Ciudad {
  id       Int       @id @default(autoincrement())
  nombre   String
  orden    Int
  zonaId   Int
  zona     Zona      @relation(fields: [zonaId], references: [id])
  clientes Cliente[]
}

// ---------- Productos y stock ----------
model Producto {
  id             Int             @id @default(autoincrement())
  codigo         String          @unique
  categoriaId    Int
  marcaId        Int
  descripcion    String          // una línea por dato
  cbmUnidad      Decimal         @default(0) @db.Decimal(8, 4)
  fotoPath       String?
  activo         Boolean         @default(true)
  posicion       Int
  categoria      Categoria       @relation(fields: [categoriaId], references: [id])
  marca          Marca           @relation(fields: [marcaId], references: [id])
  lotes          Lote[]
  ajustes        AjusteStock[]
  precios        PrecioCliente[]
  lineasProforma LineaProforma[]
  lineasFactura  LineaFactura[]
  lineasPedido   LineaPedido[]
  ventasBase     VentaBase[]
  mesesSinStock  MesSinStock[]
  periodosSinStock PeriodoSinStock[]
```

```
  @@index([categoriaId, marcaId, posicion])
}

model Lote {                      // FIFO: se descuenta del más viejo con quedan > 0
  id            Int           @id @default(autoincrement())
  productoId    Int
  pedidoId      Int?           // null = lote Inicial
  esInicial     Boolean       @default(false)
  fechaRecibido DateTime
  cantidad      Int
  quedan        Int
  fobUnidad     Decimal       @db.Decimal(14, 4)
  aduanaUnidad  Decimal       @db.Decimal(14, 4)
  costoUnidad   Decimal       @db.Decimal(14, 4)
  producto      Producto      @relation(fields: [productoId], references: [id])
  pedido        Pedido?       @relation(fields: [pedidoId], references: [id])
  consumos      LoteConsumo[]

  @@index([productoId, fechaRecibido])
}

model AjusteStock {
  id           Int      @id @default(autoincrement())
  productoId   Int
  fecha        DateTime @default(now())
  unidades     Int      // positivo entra, negativo sale
  motivo       String
  stockAntes   Int
  stockDespues Int
  producto     Producto @relation(fields: [productoId], references: [id])
}

// ---------- Clientes y precios ----------
model Cliente {
  id           Int             @id @default(autoincrement())
  codigo       String          @unique
  nombre       String
  ciudadId     Int
  direccion    String?
  contacto     String?
  diasCredito  Int             @default(0)
  aplicaIsv    Boolean         @default(false)
  activo       Boolean         @default(true)
  ciudad       Ciudad          @relation(fields: [ciudadId], references: [id])
  precios      PrecioCliente[]
  proformas    Proforma[]
  facturas     Factura[]
}

model PrecioCliente {             // precio sin ISV; cambios van a Historial
  id         Int      @id @default(autoincrement())
  clienteId  Int
  productoId Int
  precio     Decimal  @db.Decimal(14, 2)
  fecha      DateTime @default(now())
  cliente    Cliente  @relation(fields: [clienteId], references: [id])
  producto   Producto @relation(fields: [productoId], references: [id])

  @@unique([clienteId, productoId])
}

// ---------- Proformas, facturas y cobros ----------
model Proforma {
  id        Int             @id @default(autoincrement())
  numero    Int             @unique
  clienteId Int
  fecha     DateTime
```

```
  dias      Int
  vence     DateTime
  tipo      TipoVenta       @default(CREDITO)
  aplicaIsv Boolean         @default(false)
  estado    EstadoProforma  @default(PENDIENTE)
  creadoEn  DateTime        @default(now())
  cliente   Cliente         @relation(fields: [clienteId], references: [id])
  lineas    LineaProforma[]
  factura   Factura?
}

model LineaProforma {
  id             Int       @id @default(autoincrement())
  proformaId     Int
  productoId     Int
  cantidad       Int
  precio         Decimal   @db.Decimal(14, 2)
  precioCambiado Boolean   @default(false)
  tipo           TipoLinea @default(NORMAL)
  proforma       Proforma  @relation(fields: [proformaId], references: [id], onDelete:
Cascade)
  producto       Producto  @relation(fields: [productoId], references: [id])
}

model Factura {
  id          Int            @id @default(autoincrement())
  numero      Int            @unique   // interno, no es factura SAR
  proformaId  Int?           @unique
  clienteId   Int
  fecha       DateTime
  vence       DateTime
  tipo        TipoVenta
  subtotal    Decimal        @db.Decimal(14, 2)
  isv         Decimal        @default(0) @db.Decimal(14, 2)
  total       Decimal        @db.Decimal(14, 2)
  cargaInicial Boolean       @default(false)
  eliminada   Boolean        @default(false)
  eliminadaEn DateTime?
  proforma    Proforma?      @relation(fields: [proformaId], references: [id])
  cliente     Cliente        @relation(fields: [clienteId], references: [id])
  lineas      LineaFactura[]
  abonos      Abono[]

  @@index([clienteId, fecha])
  @@index([fecha])
}

model LineaFactura {
  id               Int           @id @default(autoincrement())
  facturaId        Int
  productoId       Int
  descripcion      String        // texto conservado al facturar
  cantidadOriginal Int
  cantidad         Int           // baja si se edita la factura
  precio           Decimal       @db.Decimal(14, 2)
  tipo             TipoLinea     @default(NORMAL)
  costoFifo        Decimal       @default(0) @db.Decimal(14, 4)
  factura          Factura       @relation(fields: [facturaId], references: [id], onDelete:
Cascade)
  producto         Producto      @relation(fields: [productoId], references: [id])
  consumos         LoteConsumo[]
}

model LoteConsumo {              // de qué lote salió cada pieza (para devolverla a su lote)
  id             Int          @id @default(autoincrement())
  lineaFacturaId Int
  loteId         Int
  cantidad       Int
```

```
  costoUnidad    Decimal      @db.Decimal(14, 4)
  lineaFactura   LineaFactura @relation(fields: [lineaFacturaId], references: [id],
onDelete: Cascade)
  lote           Lote         @relation(fields: [loteId], references: [id])
}

model Abono {
  id                  Int        @id @default(autoincrement())
  facturaId           Int
  fecha               DateTime
  monto               Decimal    @db.Decimal(14, 2)
  numeroRecibo        String?
  medio               MedioAbono
  chequePendienteHasta DateTime?
  factura             Factura    @relation(fields: [facturaId], references: [id])

  @@index([facturaId])
}

// ---------- Compras ----------
model Pedido {
  id              Int            @id @default(autoincrement())
  numero          Int            @unique
  estado          EstadoPedido   @default(EN_PRODUCCION)
  fechaColocado   DateTime
  llegadaAprox    DateTime?
  porcentajeAduana Decimal       @db.Decimal(5, 2)
  fobEstimado     Decimal        @db.Decimal(14, 2)
  fobReal         Decimal?       @db.Decimal(14, 2)
  proveedorPagado Boolean        @default(false)
  cbmPackingList  Decimal?       @db.Decimal(10, 4)
  recibidoEn      DateTime?
  lineas          LineaPedido[]
  pagos           PagoPedido[]
  gastos          GastoRecepcion[]
  lotes           Lote[]
}

model LineaPedido {
  id         Int      @id @default(autoincrement())
  pedidoId   Int
  productoId Int
  cantidad   Int
  fobUnidad  Decimal  @db.Decimal(14, 4)
  cbmUnidad  Decimal  @db.Decimal(8, 4)
  pedido     Pedido   @relation(fields: [pedidoId], references: [id], onDelete: Cascade)
  producto   Producto @relation(fields: [productoId], references: [id])
}

model PagoPedido {
  id       Int            @id @default(autoincrement())
  pedidoId Int
  fecha    DateTime
  tipo     TipoPagoPedido
  monto    Decimal        @db.Decimal(14, 2)   // puede ser negativo (crédito)
  nota     String?
  pedido   Pedido         @relation(fields: [pedidoId], references: [id])
}

model GastoRecepcion {
  id          Int     @id @default(autoincrement())
  pedidoId    Int
  concepto    String           // flete, seguro, impuestos, agente, almacenaje...
  monto       Decimal @db.Decimal(14, 2)
  aproximado  Boolean @default(false)
  pedido      Pedido  @relation(fields: [pedidoId], references: [id], onDelete: Cascade)
}
```

```
// ---------- Caja y gastos ----------
model CategoriaGasto {
  id          Int              @id @default(autoincrement())
  nombre      String
  grupo       GrupoGasto
  tipo        TipoGasto
  montoFijo   Decimal?         @db.Decimal(14, 2)
  activaDesde DateTime?
  activaHasta DateTime?
  esSistema   Boolean          @default(false)   // Devoluciones/cambios
  fijosMes    GastoFijoMes[]
  movimientos MovimientoGasto[]
}

model GastoFijoMes {             // los fijos se generan por mes
  id         Int            @id @default(autoincrement())
  categoriaId Int
  anio       Int
  mes        Int
  monto      Decimal        @db.Decimal(14, 2)
  real       Boolean        @default(false)      // true al cerrar el mes
  categoria  CategoriaGasto @relation(fields: [categoriaId], references: [id])

  @@unique([categoriaId, anio, mes])
}

model MovimientoGasto {          // gastos variables
  id          Int            @id @default(autoincrement())
  categoriaId Int
  fecha       DateTime
  monto       Decimal        @db.Decimal(14, 2)
  nota        String?
  categoria   CategoriaGasto @relation(fields: [categoriaId], references: [id])
}

model SaldoCaja {                // registro único (id = 1), se escribe a mano
  id              Int      @id @default(1)
  efectivo        Decimal  @default(0) @db.Decimal(14, 2)
  cheques         Decimal  @default(0) @db.Decimal(14, 2)
  bancosUsd       Decimal  @default(0) @db.Decimal(14, 2)
  bancosUsdValorL Decimal  @default(0) @db.Decimal(14, 2)
  actualizadoEn   DateTime @updatedAt
}

model Anio {
  anio                    Int      @id
  metaAnual               Decimal  @default(0) @db.Decimal(14, 2)
  porcentajeBrutoAnterior Decimal? @db.Decimal(8, 6)
  cerrado                 Boolean  @default(false)
  multiplicadoresIniciales Json?   // por categoría, solo arranque
}

model MesContable {              // PE y meta se fijan el día 1
  id      Int     @id @default(autoincrement())
  anio    Int
  mes     Int
  peMes   Decimal @db.Decimal(14, 2)
  metaMes Decimal @db.Decimal(14, 2)

  @@unique([anio, mes])
}

// ---------- Demanda y auditoría ----------
model VentaBase {                // año base 2026, editable
  id         Int      @id @default(autoincrement())
  productoId Int
  anio       Int
  mes        Int
```

```
  unidades   Int
  producto   Producto @relation(fields: [productoId], references: [id])

  @@unique([productoId, anio, mes])
}

model PeriodoSinStock {          // se abre cuando el stock del código llega a 0 y se cierra
cuando vuelve a haber piezas
  id         Int       @id @default(autoincrement())
  productoId Int
  desde      DateTime
  hasta      DateTime?              // null = sigue en cero
  producto   Producto  @relation(fields: [productoId], references: [id])

  @@index([productoId, desde])
}

model MesSinStock {              // veredicto que registra el cierre de mes a partir de
PeriodoSinStock
  id         Int      @id @default(autoincrement())
  productoId Int
  anio       Int
  mes        Int
  producto   Producto @relation(fields: [productoId], references: [id])

  @@unique([productoId, anio, mes])
}

model Historial {
  id         Int      @id @default(autoincrement())
  entidad    String
  entidadId String
  campo      String
  antes      String?
  despues    String?
  motivo     String?
  fecha      DateTime @default(now())

  @@index([entidad, entidadId])
}
```

Reglas que el modelo debe respetar

- **Operaciones con stock en transacción** (`prisma.$transaction`): ingresar pedido, editar o eliminar factura, ajustes y recepción de pedido deben descontar o devolver lotes y escribir el historial de forma atómica.
- Stock de un código = suma de `quedan` de sus lotes. Apartado = unidades en proformas pendientes. Disponible = stock − apartado (se calculan, no se guardan).
- **Mes sin stock** (definición de Javier: “mes en cero”): un código cuenta como mes sin stock si estuvo con stock 0 todo el mes calendario. Se guarda el historial de períodos (`PeriodoSinStock`) y no solo el veredicto; el cierre de mes aplica la regla y escribe `MesSinStock`. El porcentaje de días exigido (`pctMesSinStock`, por defecto 100) es configurable por si se quiere flexibilizar, por ejemplo a un mínimo de días.
- Una factura de carga inicial no tiene líneas; su monto abonado se registra como un abono.
- Nada con movimiento se borra: se inactiva (`activo = false`) o se marca (`eliminada`).
- Toda edición relevante (precios, meta, gastos pasados, facturas, ajustes, lote Inicial, cierre de año) escribe en `Historial`.
- Los repartos (gastos por CBM, FOB real por línea) dejan el residuo en la última línea para cuadrar al centavo.

10. Decisiones confirmadas y pendientes

Decisiones confirmadas

Tema	Decisión
Servidor	Ubuntu 22.04 LTS o 24.04 LTS, con Nginx y PostgreSQL 15/16. Compatible con CatalogoHN.
Node.js	Node 20 LTS (o 22 LTS). Compatible con CatalogoHN y con el ERP.
Copia externa de respaldos	Google Drive con `rclone` (costo $0). Carpeta privada de Drive; configuración en la sección 8.
Backups de DigitalOcean	Desactivados por ahora (opcional; costo extra del 20%). El `pg_dump` diario con `rclone` cubre el requisito de respaldo diario fuera del servidor.
Moneda y cálculos	`decimal.js` en el backend y `Intl.NumberFormat("es-HN", { style: "currency", currency: "HNL" })` en pantalla. Evita errores de punto flotante y permite dejar el residuo en la última línea para cuadrar al centavo.
PDFs	Se generan en el backend (`@react-pdf/renderer`, o `pdfkit` si hiciera falta), porque el respaldo semanal por correo corre sin navegador abierto. Para el usuario, el backend entrega el buffer o stream de descarga.
Excel	`exceljs` (el paquete `xlsx` público está desactualizado y tiene vulnerabilidades reportadas).
ORM	Prisma.
Despliegue	`git pull` manual en el Droplet, con PM2.
Dominio	`app.kolben.catalogohn.com` (producción) y `test.app.kolben.catalogohn.com` (pruebas).
Mes sin stock	Según Javier, “mes en cero”: código con stock 0 todo el mes calendario. Se registran períodos sin stock desde la Etapa 3; el porcentaje de días exigido es configurable (por defecto 100). Alimenta la Calculadora desde 2028 (2027 usa el año base 2026).
Datos de arranque	Javier los carga manualmente por pantalla cuando reciba el sistema (como pide la sección 8 de la spec). No se desarrolla importador; sí datos semilla para pruebas.
Facturas	Internas, sin formato fiscal del SAR.
Entregas	Dos avances a Javier: uno a la mitad y otro al terminar el sistema, ligados a sus dos pagos (sección 12).
Memoria del Droplet	Uso actual cerca del 45%: hay margen para el ERP; PM2 con límite de 512 MB por proceso.

Pendientes

#	Punto	Propuesta actual
1	Registros DNS de los subdominios	Crear un registro A para `app.kolben.catalogohn.com` y otro para `test.app.kolben.catalogohn.com` hacia la IP del Droplet.
2	Datos de ejemplo de la spec	Los números de los mockups son ilustrativos; las reglas sí valen.

#	Punto	Propuesta actual
3	Cuenta de Google para el respaldo	Definir qué cuenta de Google Drive se usará y quién la administra.
4	Cuenta gratuita de Healthchecks.io	Para los avisos de respaldo; crear una verificación por script y poner el correo de Diego.
5	Repositorio en GitHub	Crear `kolben-erp`, ramas `main` y `develop`, y proteger `main`.

11. Plan de trabajo por etapas

El plan sigue las 13 pantallas de la especificación (Entrada + pantallas 0 a 12) y agrega una etapa inicial de cimientos y una final de cierre, para un total de **15 etapas**. El orden respeta las dependencias: primero lo que alimenta a lo demás (productos, clientes), luego las ventas, después las compras y al final los tableros que leen de todo.

Los mockups de cada pantalla están en el **Anexo A** (al final del documento), con la etapa en la que se construye cada una.

Cómo se trabaja cada etapa

- Se programa en una rama `feature/etapa-N-nombre` creada desde `develop`; al terminar, merge a `develop` y despliegue a **pruebas**.
- Se prueba en el entorno de pruebas contra los criterios de “Listo cuando”. Solo si pasa, se hace merge a `main` y se despliega a producción (sección 5).
- Cada etapa incluye su migración de Prisma, su API, su pantalla, y los eventos de la sección 6 de la spec que le correspondan.
- Durante el desarrollo se carga data de prueba; la carga real (arranque, sección 8 de la spec) se hace al final, en la Etapa 14.

Etapas

Etapa	Pantalla (spec)	Qué se entrega	Depende de	Listo cuando
0	Cimientos	Repositorio y ramas; backend y frontend base; Prisma conectado; Nginx, HTTPS, PM2 y DNS en pruebas y producción; flujo de despliegue; `/api/health`; respaldo diario de base de datos y `uploads/` con aviso si falla; datos semilla para pruebas (sin correos reales).	—	Una página de prueba abre en test y en producción por HTTPS; el respaldo se restaura en pruebas.
1	Entrada + Menú y Configuración (12)	Login con JWT en cookie HttpOnly, creación del usuario en el primer uso, salir; menú lateral fijo (ancho 1280 px); datos de la proforma, bancos (orden), ISV, respaldo (correo y día), cambio de contraseña, enviar prueba; regla de mes sin stock (por defecto, todo el mes en cero); script para restablecer la contraseña desde el servidor.	0	Se entra y se sale; la configuración se guarda y persiste; sin sesión no se accede a ninguna ruta.

Etapa	Pantalla (spec)	Qué se entrega	Depende de	Listo cuando
2	Productos (8)	Categorías y marcas con logo y orden; listado por categoría y marca con filtros y buscador; alta y edición de código (foto, CBM, descripción multilínea); orden con flechas y por número; inactivar y eliminar solo sin movimiento; campos de arranque (stock, FOB, costo que crean el lote Inicial); fotos y logos se redimensionan al subirlos (`sharp`); historial.	1	El orden guardado se respeta en la lista; se crea un código con lote Inicial; el valor del inventario a costo cuadra.
3	Ficha de código (9)	Detalle con stock, apartado, disponible y valores; lotes FIFO; ajustes de stock en dos pasos con motivo; edición del lote Inicial con confirmación; servicio FIFO reutilizable; **registro de períodos sin stock** (`PeriodoSinStock`): el servicio abre un período cuando el stock del código llega a 0 (o cuando se crea con stock 0) y lo cierra cuando vuelve a haber piezas.	2	Un ajuste saca del lote más viejo y entra al más nuevo; el historial registra antes y después; un código que llega a 0 y se repone deja su período abierto y cerrado con las fechas correctas.
4	Clientes (6)	Zonas y ciudades; lista, buscador y ficha del cliente; alta, edición e inactivación; precios por cliente con historial y marca de “sin precio”. Las métricas (ritmo, ticket, margen) quedan listas pero se llenan en la Etapa 6.	1, 2	Ciudad asigna zona sola; un cliente con saldo no se inactiva; el historial de precios guarda cada cambio.
5	POS · Proformas (1)	Buscador de cliente y de códigos con disponible; líneas con precio de la lista, precio cambiado y línea Cambio; totales e ISV; %U.B. solo en pantalla; guardar proforma (aparta unidades); pendientes con alerta de 7 días; PDF al cliente con `@react-pdf/renderer`; ingresar pedido (crea factura).	2, 3, 4	Aparta y libera unidades; el PDF coincide con el mockup; ingresar saca stock FIFO y guarda el lote de origen.
6	Ventas (7)	Por cobrar (zona, ciudad, cliente), todas las ventas y cambios; abonos con recibo y medio; cheque post-fechado; editar (bajar cantidades) y quitar factura; PDF de factura; factura inicial y corregir saldo; respaldo semanal de Por cobrar por correo.	5	Editar o eliminar devuelve las piezas a su lote y recalcula el mes de la factura; el PDF semanal llega al correo de prueba.
7	Ventas por código (2)	Tabla por categoría, marca y código con unidades Ene–Dic; año base 2026 editable con historial; desde 2027 sale de las facturas (sin Cambios) y se marcan visualmente los meses sin stock; selector de año.	6	Los totales coinciden con las facturas; el año base se edita y se guarda; un mes en cero aparece marcado.

Etapa	Pantalla (spec)	Qué se entrega	Depende de	Listo cuando
8	Utilidad por código (3)	Drill-down año → categoría → marca → código con unidades, ingreso, costo FIFO, utilidad y %U.B.; marcas ordenadas por utilidad.	6	La utilidad coincide con la suma de facturas; Cambios y cargas iniciales no cuentan.
9	Compras · Pedidos (5)	Pedidos activos con estados; pagos al proveedor y gastos; “Proveedor pagado completo”; marcar recibido con gastos reales, CBM y costeo por código; creación de lotes; bloqueos y avisos; historial y descargable Excel por pedido.	2, 3	Al recibir, el costo total de los lotes cuadra al centavo con FOB real + gastos; el stock entra y el pedido sale de “en camino”.
10	Compras · Calculadora (4)	Stock extra y ciclo de pedidos; simulación mes a mes por código; demanda con regla de mes sin stock (se sustituye por el promedio de los meses con stock del mismo código; sin datos = 0); calculado, ajuste, final y FOB total; totales contra el disponible; PDF y Excel de cotización; “Ingresar orden nueva” que crea el pedido.	7, 9	El cálculo coincide con las fórmulas de la spec en casos de prueba, incluido un código con un mes en cero; la orden creada aparece en Pedidos.
11	Caja (10)	Saldos manuales; deuda por pedido y disponible para compras; meta anual, % bruto y multiplicadores iniciales; flujo de 12 meses con gastos fijos y variables; Devoluciones/cambios automáticas; PE y meta mensual; cerrar y reabrir año; corrección de meses pasados con historial; respaldo semanal de Caja (CSV).	6, 9	U.N.1 y U.N.2 cuadran con ventas y gastos; el cierre de mes fija PE y meta; editar un gasto fijo respeta “solo este mes / en adelante”.
12	Resumen (0)	Tablero de solo lectura: ventas, U.B., U.N.1, U.N.2 y balance; gráfica mensual con PE y meta; barras anuales; por cobrar, inventario, multiplicadores, prueba ácida, posición financiera y pedidos en camino; selector de año.	6, 9, 11	Cada cifra coincide con su pantalla de origen; cambia en tiempo real al registrar una venta o un pago.
13	Catálogo (11)	Vista previa en cuadrícula; portada subida; paleta; PDF con 6 fichas por página según el orden de Productos; solo códigos activos con disponible mayor o igual a 1.	2	El PDF respeta el orden, sin precios ni títulos; un código sin disponible desaparece.
14	Cierre, arranque y salida	Procesos automáticos en servidor (cierre de mes, que también registra los meses sin stock desde los períodos; respaldo semanal); pruebas de punta a punta con datos reales de ejemplo; revisión de permisos y archivos; guía de carga inicial para que Javier cargue todo por pantalla en producción (sección	0–13	Se factura una venta completa de principio a fin; el respaldo se restaura; Javier firma de conformidad.

Etapa	Pantalla (spec)	Qué se entrega	Depende de	Listo cuando
		8 de la spec, sin importador); capacitación a Javier; documento de entrega.		

Nota: al terminar las Etapas 6 y 9 ya se puede operar ventas y compras; las Etapas 7, 8, 12 y 13 son lectura y reportes, y pueden reordenarse según lo que Javier necesite primero. Los tiempos por etapa no están incluidos: se estiman una vez confirmado el orden. Los dos avances a Javier se explican en la sección 12.

12. Antes de comenzar y entregas

Lista de verificación previa

Punto	Estado
Memoria del Droplet	Listo: uso actual cerca del 45%. Vigilar con `pm2 monit` al generar el catálogo por primera vez.
Límite de memoria en PM2	Listo: 512 MB por proceso (sección 6).
Repositorio en GitHub, ramas `main` y `develop`, `main` protegida	Pendiente (Etapa 0)
DNS de `app.` y `test.app.`	Pendiente (Etapa 0)
Cuenta de Google Drive y `rclone`	Pendiente (Etapa 0)
Cuenta gratuita de Healthchecks.io	Pendiente (Etapa 0)
Datos semilla para pruebas	Se hacen en la Etapa 0 y se amplían con cada etapa. Nunca con el correo real de Javier.
Restablecer contraseña	Script en el servidor, Etapa 1.
Facturas internas (no SAR)	Confirmado.

Carga inicial de datos

Javier carga todo manualmente por pantalla cuando se le entregue el sistema; no hay importador. Por eso las pantallas de arranque deben quedar cómodas (los campos especiales del producto que crean el lote Inicial, la factura inicial y la corrección de saldo). Se prueban completas en la Etapa 14. Orden de carga según la sección 8 de la spec:

Paso	Qué se carga	Dónde
1	Usuario y Configuración (proforma, bancos, ISV, respaldo)	Entrada y Configuración
2	Categorías y marcas	Productos · Listas
3	Productos con stock inicial, FOB/u y costo/u (crea el lote Inicial)	Productos
4	Zonas, ciudades, clientes y precios	Clientes
5	Facturas por cobrar viejas (factura inicial)	Ventas
6	Ventas por código 2026 (año base)	Ventas por código
7	Pedidos abiertos	Calculadora y Pedidos

Paso	Qué se carga	Dónde
8	Saldos, meta anual, % bruto 2026 y multiplicadores iniciales	Caja
9	Primera venta (el número de la proforma sigue a la última factura real)	POS

Avances a Javier

Se le muestran dos avances en el entorno de pruebas, ligados a sus dos pagos. Propuesta de corte (se puede mover):

Avance	Cuándo	Qué puede ver y probar
Avance 1 (mitad)	Al terminar la Etapa 6	Entrar al sistema, configurar, cargar productos y clientes con sus precios, hacer proformas e ingresarlas como facturas, ver Por cobrar y registrar abonos, y ver cómo se mueve el inventario por lotes.
Avance 2 (final)	Al terminar la Etapa 14	El sistema completo: compras y calculadora, caja, resumen, catálogo, respaldos y la carga inicial en producción.

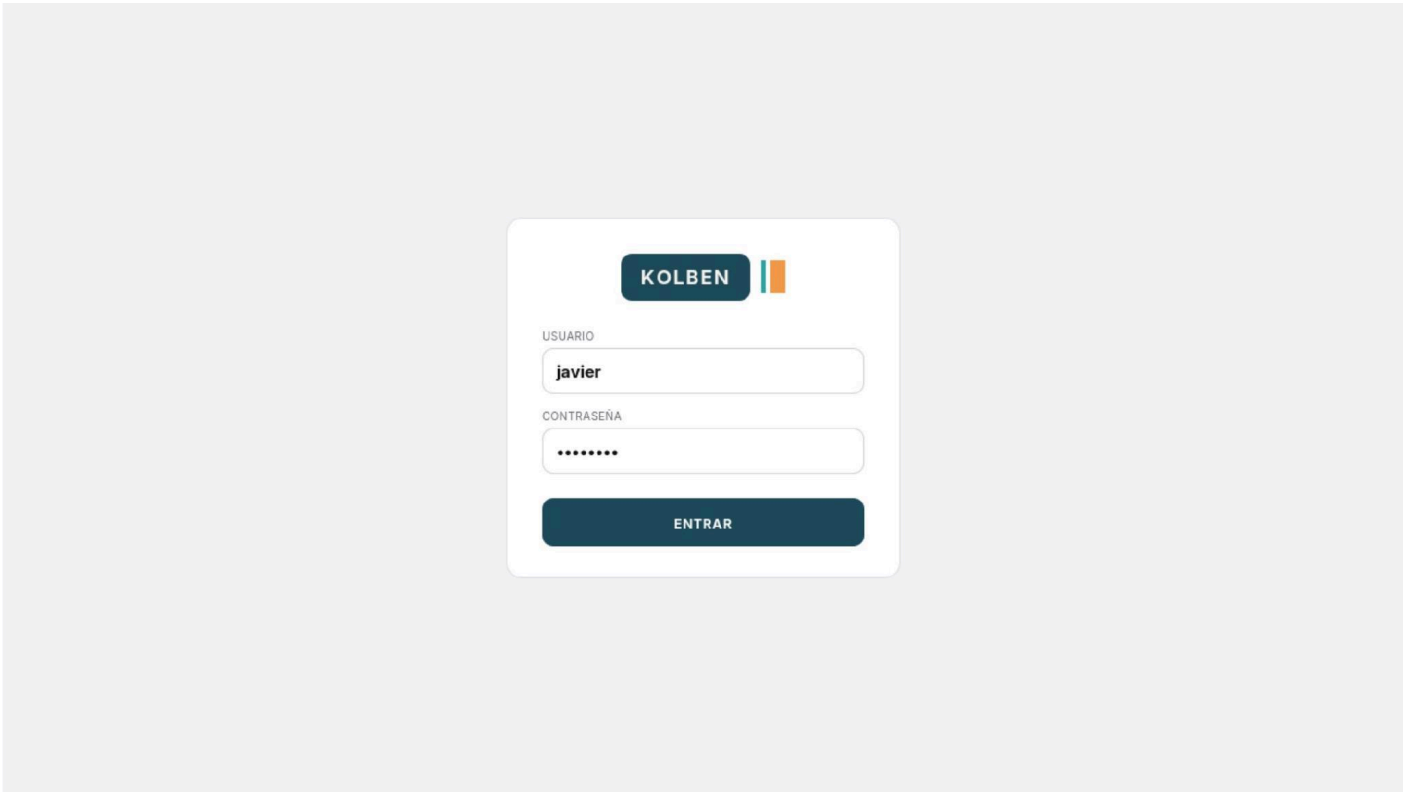
- Cada avance se pasa primero a pruebas (`test.app.kolben.catalogohn.com`) y Javier lo revisa ahí.
- Sus observaciones se anotan en una lista (por ejemplo en GitHub Issues o Projects) y se separan en correcciones de lo ya especificado y cambios nuevos fuera de la spec; estos últimos se acuerdan aparte antes de programarlos.
- Cuando Javier da su visto bueno, se pasa a `main` y a producción, con respaldo previo (sección 5).
- Se recomienda dejar por escrito qué incluye cada avance antes de presentarlo.

Anexo A. Mockups de la especificación

Mockups de alta fidelidad tomados de la Especificación funcional v1.0 (23/10/2026). Los números que muestran son de ejemplo; lo que manda son las reglas y fórmulas de la spec. Los bloques “Estado siguiente” muestran lo que aparece al tocar cada botón. Cada pantalla indica la etapa del plan (sección 11) en la que se construye.

A.1 Entrada (login) · Etapa 1

Pantalla de entrada



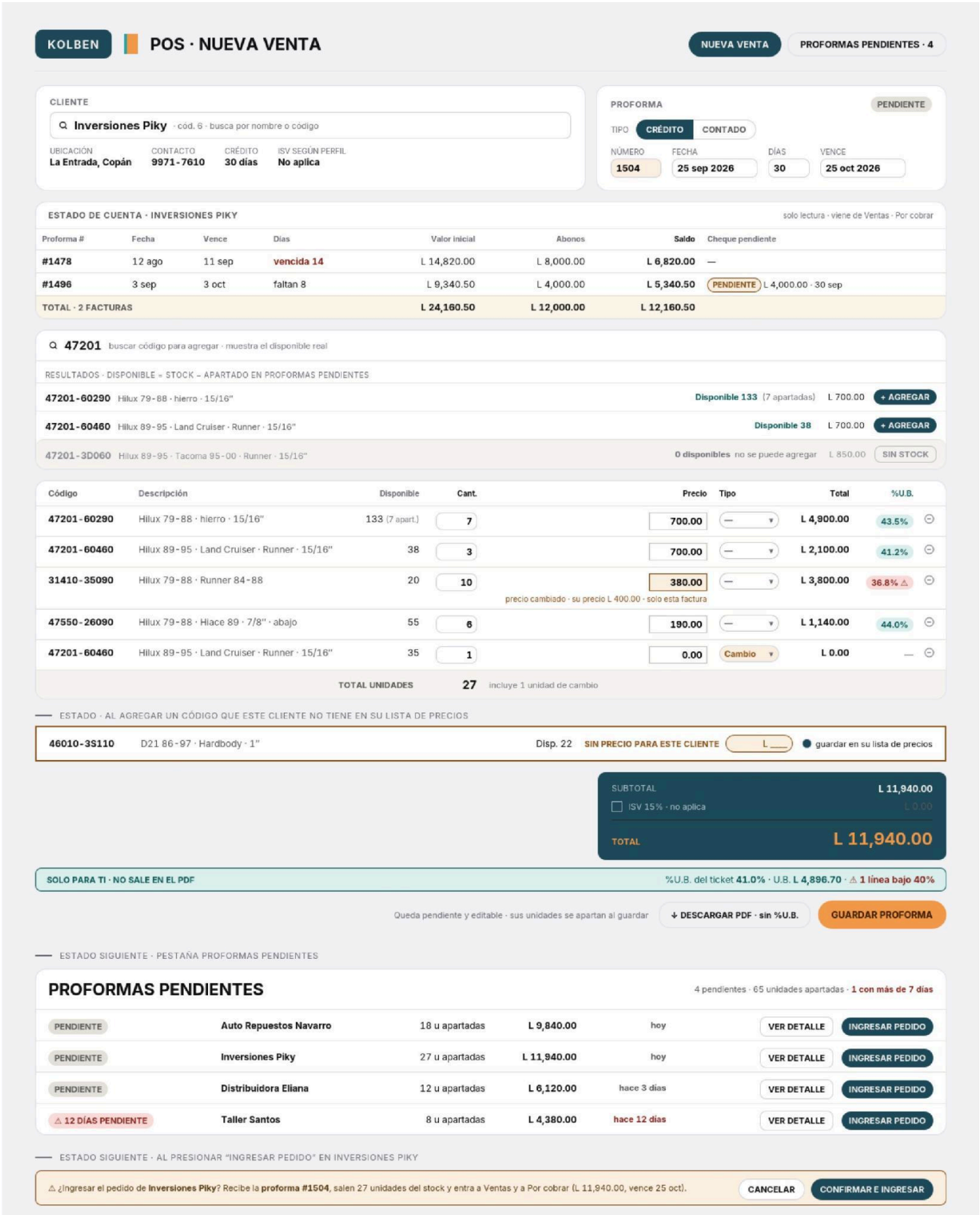
A.2 Pantalla 0 · Resumen · Etapa 12

Resumen



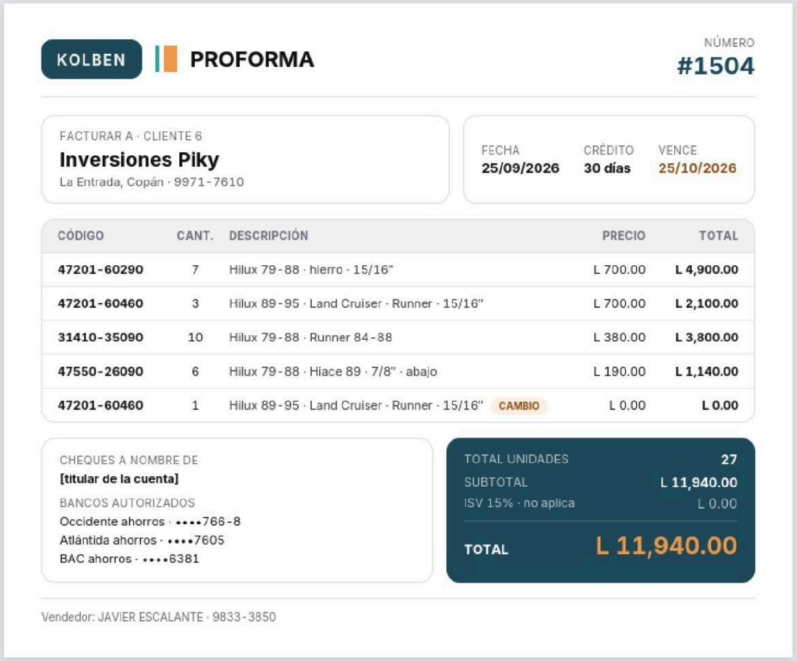
A.3 Pantalla 1 · POS · Etapa 5

POS y estados siguientes



PDF de la proforma al cliente

ESTADO SIGUIENTE · AL PRESIONAR "DESCARGAR PDF" · LO QUE RECIBE EL CLIENTE (SIN %U.B.)



KOLBEN **PROFORMA**

NÚMERO **#1504**

FACTURAR A · CLIENTE 6

Inversiones Piky

La Entrada, Copán · 9971-7610

FECHA	CRÉDITO	VENCE
25/09/2026	30 días	25/10/2026

CÓDIGO	CANT.	DESCRIPCIÓN	PRECIO	TOTAL
47201-60290	7	Hilux 79-88 · hierro · 15/16"	L 700.00	L 4,900.00
47201-60460	3	Hilux 89-95 · Land Cruiser · Runner · 15/16"	L 700.00	L 2,100.00
31410-35090	10	Hilux 79-88 · Runner 84-88	L 380.00	L 3,800.00
47550-26090	6	Hilux 79-88 · Hiace 89 · 7/8" · abajo	L 190.00	L 1,140.00
47201-60460	1	Hilux 89-95 · Land Cruiser · Runner · 15/16" CAMBIO	L 0.00	L 0.00

CHEQUES A NOMBRE DE

[titular de la cuenta]

BANCOS AUTORIZADOS

Occidente ahorros · ••••766-8

Atlántida ahorros · ••••7605

BAC ahorros · ••••6381

TOTAL UNIDADES	27
SUBTOTAL	L 11,940.00
ISV 15% · no aplica	L 0.00
TOTAL	**L 11,940.00**

Vendedor: JAVIER ESCALANTE · 9833-3850

A.4 Pantalla 2 · Ventas por código · Etapa 7

Ventas por código y edición del año base

KOLBEN | VENTAS POR CÓDIGO — 2026

001 M. FRENO — 10 CÓDIGOS

CÓDIGO	Ene	Feb	Mar	Abr	May	Jun	Jul	Ago	Sep	Oct	Nov	Dic	Anual
TOYOTA 10 códigos · 1,101 unidades													
47201-60290	49	47	14	21	–	–	58	28	6	–	–	–	223
47201-60460	78	37	–	–	–	–	94	12	6	–	–	–	227
47201-3D060	38	20	32	6	6	10	51	6	5	–	–	–	174
47201-3D050	9	4	–	6	31	–	28	4	–	–	–	–	82
47201-09210	12	3	–	–	7	–	3	2	2	–	–	–	29
47201-0K040	12	6	–	4	7	4	6	–	2	–	–	–	41
47201-04060	29	12	5	4	–	–	60	–	–	–	–	–	110
47201-04150	23	3	10	2	6	2	6	6	14	–	–	–	72
47201-12740	39	9	30	–	8	6	1	–	–	–	–	–	93
47201-12250	26	4	–	20	–	–	–	–	–	–	–	–	50
SUBTOTAL	**315**	**145**	**91**	**63**	**65**	**22**	**307**	**58**	**35**	–	–	–	**1,101**

002 M. CLUTCH ›

003 AUX DE RUEDA ›

004 AUX DE CLUTCH ›

ESTADO SIGUIENTE · ELEGIR 2026 (AÑO BASE) Y "✎ EDITAR"

2026 · AÑO BASE · EDITANDO — CANCELAR | GUARDAR

el único año editable · desde 2027 todo sale de las facturas y es solo lectura

001 · M. FRENO · TOYOTA

Código	Ene	Feb	Mar	Abr	May	Jun	Jul	Ago	Sep	Oct	Nov	Dic	Año
47201-60290	0	0	0	0	0	0	0	0	0	0	0	0	0
47201-60460	18	22	15	14	0	0	0	0	0	0	0	0	69
47201-3D060	0	0	0	0	0	0	0	0	0	0	0	0	0
47201-3D050	0	0	0	0	0	0	0	0	0	0	0	0	0

+ todos los códigos activos, por categoría y marca · todo arranca en 0

A.5 Pantalla 3 · Utilidad por código · Etapa 8

Utilidad por código

KOLBEN | UTILIDAD POR CÓDIGO

2026

001 M. FRENO

INGRESO	UTILIDAD	%U.B.
L 814,790.00	L 336,450.18	41.3%

TOYOTA — 10 códigos · L 336,450.18 utilidad

CÓDIGO	Unidades	Ingreso	Costo	Utilidad	%U.B.
47201-60460	227	L 158,900.00	L 89,730.83	L 69,169.17	43.5%
47201-60290	223	L 162,790.00	L 97,600.64	L 65,187.36	40.0%
47201-3D060	174	L 121,800.00	L 70,182.90	L 51,617.10	42.4%
47201-04060	110	L 88,000.00	L 52,673.50	L 35,326.50	40.1%
47201-12740	93	L 65,100.00	L 35,761.29	L 29,338.71	45.1%
47201-3D050	82	L 57,400.00	L 33,131.28	L 24,268.72	42.3%
47201-04150	72	L 64,800.00	L 43,766.64	L 21,033.36	32.5%
47201-12250	50	L 40,000.00	L 22,896.50	L 17,103.50	42.8%
47201-0K040	41	L 32,800.00	L 18,775.13	L 14,024.87	42.8%
47201-09210	29	L 23,200.00	L 13,821.11	L 9,378.89	40.4%
SUBTOTAL TOYOTA	1,101	L 814,790.00	L 478,339.82	L 336,450.18	41.3%

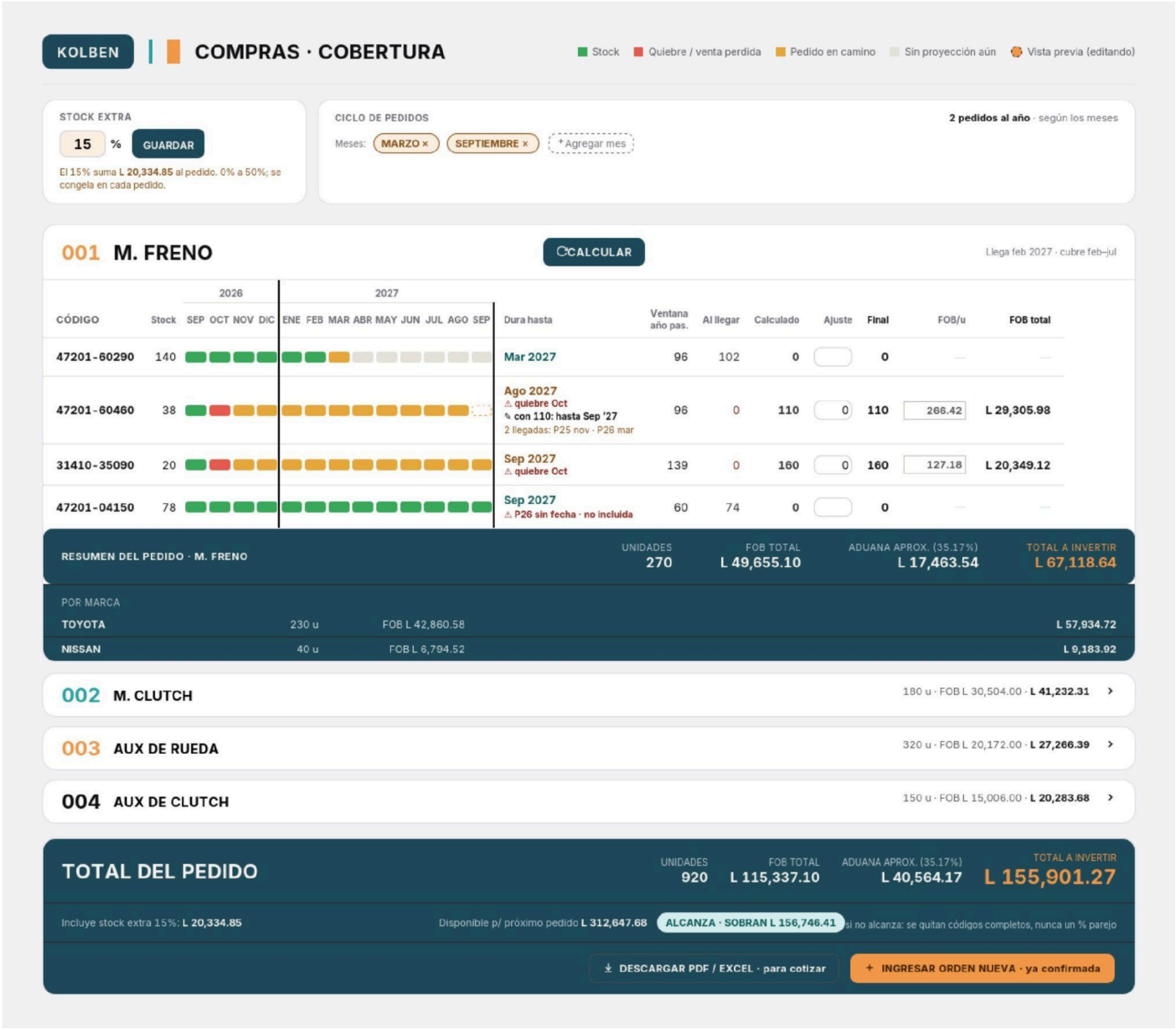
002 M. CLUTCH ›

003 AUX DE RUEDA ›

004 AUX DE CLUTCH ›

A.6 Pantalla 4 · Compras · Calculadora · Etapa 10

Calculadora de pedido



A.7 Pantalla 5 · Compras · Pedidos · Etapa 9

Pedidos, historial y recepción

KOLBEN **COMPRAS · PEDIDOS**

PEDIDOS ACTIVOS · 2

PEDIDO 25 EN CAMINO

COLOCADO	LLEGADA APROX.	VALOR FOB	ADUANA APROX. 35%	TOTAL PEDIDO
14 jun 2026	25 nov 2026 ✎	L 78,408.00 estimado · real al pagar todo	L 27,442.80 estimado 35%	L 105,850.80 FOB + aduana

↓ DESCARGAR ▲

PAGOS · 4 — + AGREGAR PAGO

18 jun	PROVEEDOR	depósito 30%	23,522.40	✎ ⊖
30 jul	PROVEEDOR	abono	44,096.00	✎ ⊖
2 ago	PROVEEDOR	crédito · pieza dañada ped. 24	−1,210.40	✎ ⊖
15 sep	GASTOS	anticipo agente aduanal	1,000.00	✎ ⊖

	PAGADO	POR PAGAR
Proveedor	66,408.00	12,000.00
Aduana y gastos	1,000.00	26,442.80
TOTAL	67,408.00	38,442.80

14 códigos · 920 u · **ver / editar ›** MARCAR RECIBIDO

✓ PROVEEDOR PAGADO COMPLETO

PEDIDO 26 EN PRODUCCIÓN

COLOCADO	LLEGADA APROX.	VALOR FOB	ADUANA APROX. 35%	TOTAL PEDIDO
2 sep 2026	15 mar 2027 ✎	L 48,708.00 estimado · real al pagar todo	L 17,047.80 estimado 35%	L 65,755.80 FOB + aduana

↓ DESCARGAR ▼

TOTAL · 2 PEDIDOS — FOB **L 127,116.00** aduana aprox. **L 44,490.60** total **L 171,606.60** por pagar **L 89,586.20** → Caja

HISTORIAL DE PEDIDOS — 24 recibidos · solo lectura · ver todos ›

PEDIDO	COLOCADO	RECIBIDO	FOB PAGADO	GASTOS REALES	% ADUANA REAL	VS 35%	DETALLE
24	nov 2025	18 may 2026	L 71,250.00	24,866.25	34.9%	−0.1	↓ DESCARGAR
23	may 2025	20 nov 2025	L 66,480.00	24,150.10	36.3%	+1.3	↓ DESCARGAR
22	nov 2024	16 may 2025	L 58,900.00	20,026.00	34.0%	−1.0	↓ DESCARGAR
21	may 2024	22 nov 2024	L 61,320.00	21,953.56	35.8%	+0.8	↓ DESCARGAR
20	nov 2023	14 may 2024	L 54,700.00	19,036.00	34.8%	−0.2	↓ DESCARGAR
PROMEDIO · ÚLTIMOS 5 PEDIDOS					35.2%	+0.2	

CONTENIDO DEL DESCARGABLE · VARIACIÓN DE COSTO · PEDIDO 25 YA RECIBIDO

	1 · COSTO ACTUAL · PEDIDO 25 (L/u)				2 · COSTO ANTERIOR · ÚLTIMO LOTE DEL CÓDIGO (L/u)				3 · VARIACIÓN (L/u)				
CÓDIGO	CANT.	FOB	ADUANA	COSTO	PEDIDO	FOB	ADUANA	COSTO	FOB	ADUANA	TOTAL	%	RESULTADO
47201-60460	110	262.09	47.44	309.53	24	205.00	43.10	248.10	+57.09	+4.34	+61.43	+24.8%	▲ SUBIÓ
31410-35090	160	125.11	40.53	165.64	24	122.00	36.90	158.90	+3.11	+3.63	+6.74	+4.2%	▲ SUBIÓ
47550-35170	90	115.00	26.20	141.20	21	115.00	26.20	141.20	0.00	0.00	0.00	0.0%	= SE MANTUVO
31470-35140	120	78.50	17.60	96.10	23	80.00	18.40	98.40	−1.50	−0.80	−2.30	−2.3%	▼ BAJÓ
47550-04030	40	188.40	44.10	232.50	sin lote anterior								★ NUEVO

+ 9 códigos más · en el mismo orden de la orden y la factura

ESTADO SIGUIENTE · AL HACER CLIC EN "+ AGREGAR PAGO"

FECHA: 20 sep 2026 — TIPO: PROVEEDOR / GASTOS — MONTO L: 12,000.00 — NOTA (OPCIONAL): saldo final proveedor — CANCELAR — GUARDAR

ESTADO SIGUIENTE · AL HACER CLIC EN "MARCAR RECIBIDO" · PEDIDO 25

Recepción (continuación)



A.8 Pantalla 6 · Clientes · Etapa 4

Clientes y ficha

KOLBEN | CLIENTES

14 clientes activos | LISTAS | + NUEVO CLIENTE

Buscar cliente por nombre, código o ciudad... — escribe y elige · abre su ficha

#	CLIENTE	CIUDAD · ZONA	COMPRA 2026	U.B. 2026 ▾	%U.B.	SALDO
1	**Inversiones Y Carwash Tabora**	La Entrada · Copán	L 412,000.00	**L 199,166.57**	**48.3%**	L 20,113.50
2	**Auto Repuestos Navarro**	Catacamas · Olancho	L 368,500.00	**L 171,815.83**	**46.6%**	L 16,620.00
3	**Repuestos Automotrices Dago**	SPS · Cortés	L 331,200.00	**L 148,742.15**	**44.9%**	L 21,300.00
4	**Distribuidora Eliana**	Catacamas · Olancho	L 254,800.00	**L 121,116.71**	**47.5%**	L 9,860.00
5	**Auto Repuestos Pinto**	Santa Rita · Copán	L 201,400.00	**L 98,172.61**	**48.7%**	L 8,400.00
6	**Auto Repuestos Bonin**	Santa Bárbara · Santa Bárbara	L 176,900.00	**L 82,123.87**	**46.4%**	L 7,600.00
7	**Repuestos y Taller López**	Corquín · Copán	L 152,300.00	**L 75,314.69**	**49.5%**	L 6,200.00
8	**Auto Repuestos Galdamez**	Santa Rosa · Copán	L 131,600.00	**L 63,086.03**	**47.9%**	—
9	**Inversiones Piky**	La Entrada · Copán	L 118,700.00	**L 54,865.57**	**46.2%**	L 12,160.50
10	**LS Centro de Servicio**	Copan Ruinas · Copán	L 96,400.00	**L 47,282.13**	**49.0%**	—
11	**Auto Repuestos Toñito**	La Entrada · Copán	L 88,300.00	**L 40,012.05**	**45.3%**	—
12	**Auto Repuestos Raúl**	La Entrada · Copán	L 72,600.00	**L 28,721.45**	**39.6% ⚠**	L 7,450.00
13	**Inversiones Aguilar**	San Nicolás · Copán	L 54,150.05	**L 25,138.56**	**46.4%**	L 5,680.00
14	**Taller Santos**	Jesús de Otoro · Intibucá	L 61,400.00	**L 23,918.80**	**39.0% ⚠**	—
TOTAL 2026 · 14 CLIENTES			**L 2,520,250.05**	**L 1,179,477.02**	**46.8%**	

DEBERÍAN PEDIR ESTE MES

5 clientes · según su ritmo normal de compra

CLIENTE	Ciudad	Compra cada	Última	Fecha esperada	Ticket promedio	
COPÁN						
Auto Repuestos Galdamez	Santa Rosa	~40 días	2 jul	**~11 ago**	L 24,600	ATRASADO
LS Centro de Servicio	Copan Ruinas	~45 días	28 jul	**~11 sep**	L 18,900	YA DEBERÍA PEDIR
Auto Repuestos Toñito	La Entrada	~35 días	12 ago	**~16 sep**	L 15,300	YA DEBERÍA PEDIR
INTIBUCÁ						
Taller Santos	Jesús de Otoro	~50 días	5 jul	**~24 ago**	L 12,400	ATRASADO
SANTA BÁRBARA						
Auto Repuestos Bonin	Santa Bárbara	~60 días	29 jul	**~27 sep**	L 27,750	LE TOCA ~27 SEP

› 3 clientes sin comprar hace mucho · más de 3 veces su ritmo — Ver ›

POR ZONA · 2026

#	ZONA	COMPRA	U.B. ▾	%U.B.
1	**Copán**	L 1,327,450.05	**L 631,759.66**	47.6%
2	**Olancho**	L 623,300.00	**L 292,932.54**	47.0%
3	**Cortés**	L 331,200.00	**L 148,742.15**	44.9%
4	**Santa Bárbara**	L 176,900.00	**L 82,123.87**	46.4%
5	**Intibucá**	L 61,400.00	**L 23,918.80**	39.0%

— ESTADO SIGUIENTE · AL ESCRIBIR "NAV" Y ELEGIR AUTO REPUESTOS NAVARRO · LA FICHA OCUPA LA PANTALLA

← VOLVER A CLIENTES

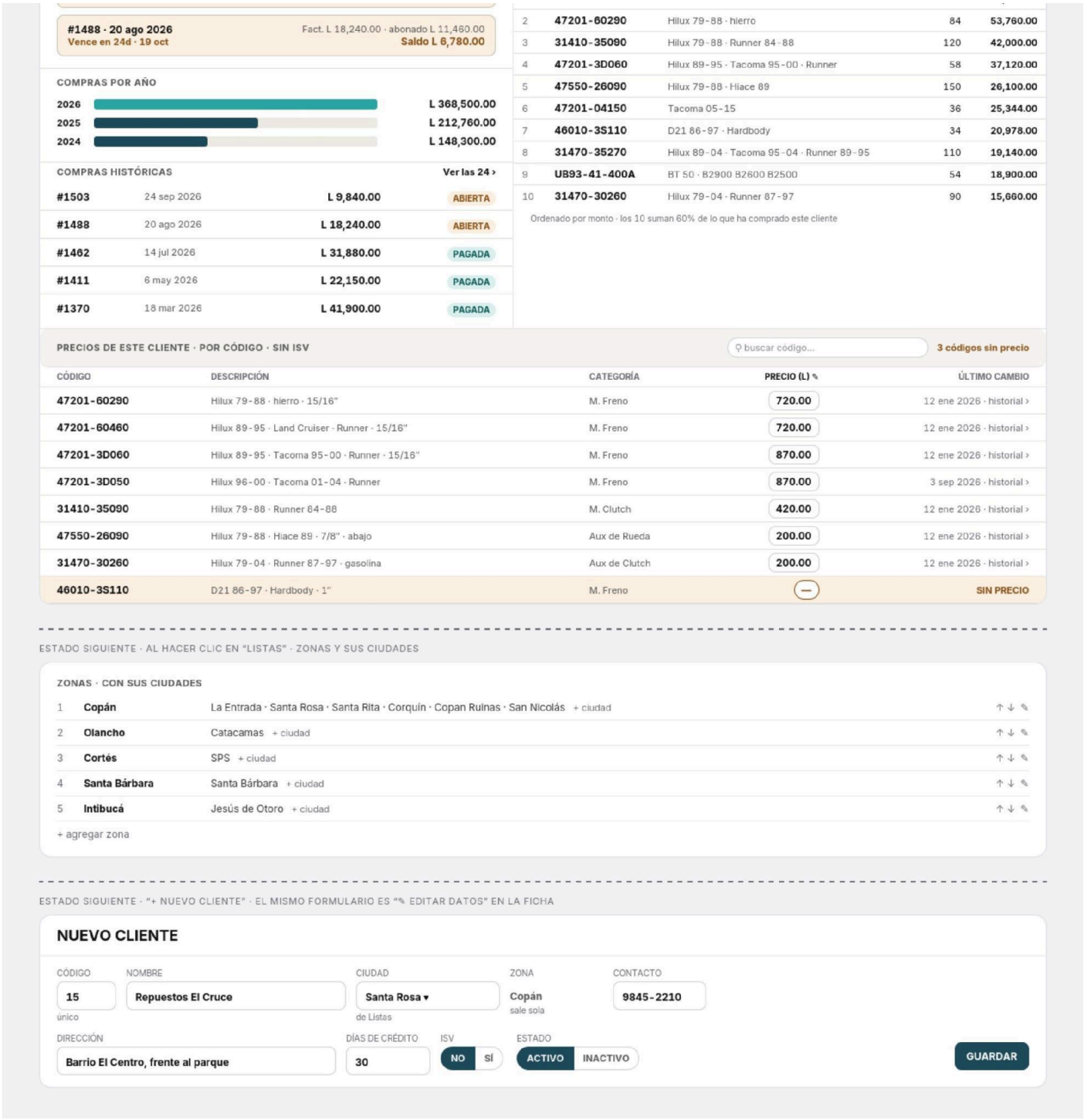
AUTO REPUESTOS NAVARRO

Cód. 1 · Catacamas, Olancho · 9832-1315 · Crédito 60 días · ISV No

✎ EDITAR DATOS

Cliente desde 2024

Compra cada **~52 días** | Última **24 sep** | Próxima **~15 nov** | AL DÍA | Ticket promedio **L 35,284** · 2026


#1488 · 20 ago 2026
Vence en 24d · 19 oct
Fact. L 18,240.00 · abonado L 11,460.00
Saldo L 6,780.00
COMPRAS POR AÑO
2026 L 368,500.00
2025 L 212,760.00
2024 L 148,300.00
COMPRAS HISTÓRICAS
Ver las 24 ›
#1503 24 sep 2026 L 9,840.00 ABIERTA
#1488 20 ago 2026 L 18,240.00 ABIERTA
#1462 14 jul 2026 L 31,880.00 PAGADA
#1411 6 may 2026 L 22,150.00 PAGADA
#1370 18 mar 2026 L 41,900.00 PAGADA
2 47201-60290 Hilux 79-88 · hierro 84 53,760.00
3 31410-35090 Hilux 79-88 · Runner 84-88 120 42,000.00
4 47201-3D060 Hilux 89-95 · Tacoma 95-00 · Runner 58 37,120.00
5 47550-26090 Hilux 79-88 · Hiace 89 150 26,100.00
6 47201-04150 Tacoma 05-15 36 25,344.00
7 46010-3S110 D21 86-97 · Hardbody 34 20,978.00
8 31470-35270 Hilux 89-04 · Tacoma 95-04 · Runner 89-95 110 19,140.00
9 UB93-41-400A BT 50 · B2900 B2600 B2500 54 18,900.00
10 31470-30260 Hilux 79-04 · Runner 87-97 90 15,660.00
Ordenado por monto · los 10 suman 60% de lo que ha comprado este cliente
PRECIOS DE ESTE CLIENTE · POR CÓDIGO · SIN ISV
buscar código...
3 códigos sin precio
CÓDIGO DESCRIPCIÓN CATEGORÍA PRECIO (L) ÚLTIMO CAMBIO
47201-60290 Hilux 79-88 · hierro · 15/16" M. Freno 720.00 12 ene 2026 · historial ›
47201-60460 Hilux 89-95 · Land Cruiser · Runner · 15/16" M. Freno 720.00 12 ene 2026 · historial ›
47201-3D060 Hilux 89-95 · Tacoma 95-00 · Runner · 15/16" M. Freno 870.00 12 ene 2026 · historial ›
47201-3D050 Hilux 96-00 · Tacoma 01-04 · Runner M. Freno 870.00 3 sep 2026 · historial ›
31410-35090 Hilux 79-88 · Runner 84-88 M. Clutch 420.00 12 ene 2026 · historial ›
47550-26090 Hilux 79-88 · Hiace 89 · 7/8" · abajo Aux de Rueda 200.00 12 ene 2026 · historial ›
31470-30260 Hilux 79-04 · Runner 87-97 · gasolina Aux de Clutch 200.00 12 ene 2026 · historial ›
46010-3S110 D21 86-97 · Hardbody · 1" M. Freno — SIN PRECIO
ESTADO SIGUIENTE · AL HACER CLIC EN "LISTAS" · ZONAS Y SUS CIUDADES
ZONAS · CON SUS CIUDADES
1 Copán La Entrada · Santa Rosa · Santa Rita · Corquín · Copan Ruinas · San Nicolás + ciudad
2 Olancho Catacamas + ciudad
3 Cortés SPS + ciudad
4 Santa Bárbara Santa Bárbara + ciudad
5 Intibucá Jesús de Otoro + ciudad
+ agregar zona
ESTADO SIGUIENTE · "+ NUEVO CLIENTE" · EL MISMO FORMULARIO ES "✎ EDITAR DATOS" EN LA FICHA
NUEVO CLIENTE
CÓDIGO 15 único
NOMBRE Repuestos El Cruce
CIUDAD Santa Rosa ▾ de Listas
ZONA Copán sale sola
CONTACTO 9845-2210
DIRECCIÓN Barrio El Centro, frente al parque
DÍAS DE CRÉDITO 30
ISV NO SÍ
ESTADO ACTIVO INACTIVO
GUARDAR

A.9 Pantalla 7 · Ventas · Etapa 6

Por cobrar, todas las ventas y cambios

KOLBEN | VENTAS

TOTAL PENDIENTE
L 115,384.00

VER: POR COBRAR ˅ | ZONA: TODAS ˅ | CIUDAD: elige zona ˅ | + FACTURA INICIAL · solo en arranque | ⚲ buscar cliente... (ej. Navarro)

Cliente	Factura	Vence	Monto inicial	%U.B.	Abonado	Nº recibo	Pendiente	
OLANCHO								
CATACAMAS								
⬇ Auto Repuestos Navarro	20 ago	19 oct (2d)	L 18,240.00	43.5%	L 11,460.00	R-0831 · R-0847	L 6,780.00	⊖
⬇ Auto Repuestos Navarro	24 sep	23 nov	L 9,840.00	43.1%	L 0.00	—	L 9,840.00	⊖
⬇ Distribuidora Eliana	15 sep	15 oct (-2d)	L 9,860.00	42.6%	L 0.00	—	L 9,860.00	⊖
TOTAL PENDIENTE · OLANCHO								L 26,480.00
SANTA BÁRBARA								
SANTA BÁRBARA								
⬇ Auto Repuestos Bonin	25 ago	24 oct	L 7,600.00	38.6% ⚠	L 0.00	—	L 7,600.00	⊖
COPÁN								
LA ENTRADA								
⬇ Inversiones y Carwash Tabora	6 ago	5 oct (-12d)	L 20,113.50	44.1%	L 0.00	—	L 20,113.50	⊖
⬇ Inversiones Piky	12 ago	11 sep (-36d)	L 14,820.00	41.5%	L 8,000.00	R-0839	L 6,820.00	⊖
⬇ Inversiones Piky	3 sep	3 oct (-14d)	L 9,340.50	42.0%	L 4,000.00 PENDIENTE	R-0850	L 5,340.50	⊖
⬇ Auto Repuestos Raúl	10 sep	10 oct (-7d)	L 12,450.00	42.8%	L 5,000.00 PENDIENTE	R-0851	L 7,450.00	⊖
CORQUÍN								
⬇ Repuestos y Taller López	17 abr	17 jun (-122d)	L 15,200.00	40.3%	L 9,000.00	R-0792	L 6,200.00	⊖
SANTA RITA								
⬇ Auto Repuestos Pinto	17 abr	17 may (-153d)	L 14,900.00	45.0%	L 6,500.00	R-0765	L 8,400.00	⊖
SAN NICOLÁS								
⬇ Inversiones Aguilar	22 abr	22 may (-148d)	L 5,680.00	39.4% ⚠	L 0.00	—	L 5,680.00	⊖
TOTAL PENDIENTE · COPÁN								L 60,004.00
CORTÉS								
SPS								
⬇ Repuestos Automotrices Dago	22 ago	21 sep (-26d)	L 21,300.00	42.0%	L 0.00	—	L 21,300.00	⊖

ESTADO SIGUIENTE · AL HACER CLIC EN ⊖ JUNTO A UNA FACTURA

⚠ Eliminar **1 factura** · Auto Repuestos Raúl · R-0851 · L 7,450.00. Es una devolución completa: todo vuelve al stock (a su lote), sale de Por cobrar y ventas y utilidad se recalculan en el mes original; queda en el historial. No se puede deshacer. | CANCELAR | CONFIRMAR ELIMINACIÓN

ESTADO SIGUIENTE · DESPLEGABLE EN "TODAS LAS VENTAS"

VER: TODAS LAS VENTAS ▾ | ESTE MES | ESTE AÑO | AÑO ANTERIOR | ‹ SEPTIEMBRE 2026 › | buscar número o cliente · en todas las fechas

Proforma #	Fecha	Cliente · zona	Unid.	Total	%U.B.	Estado	
#1503	24 sep	Auto Repuestos Navarro · Catacamas	18	L 9,840.00	43.1%	SALDO L 9,840.00 · vence 24 oct	PDF VER
#1502	22 sep	Inversiones Piky · La Entrada	27	L 10,517.36	41.0%	PAGADA	PDF VER
#1501	18 sep	Auto Repuestos Pinto · Santa Rita	8	L 4,380.00	38.9% ⚠	PAGADA · contado	PDF VER
#1500	15 sep	Distribuidora Eliana · Catacamas	22	L 9,860.00	42.6%	SALDO L 9,860.00 · vence 15 oct	PDF VER
#1499	10 sep	Auto Repuestos Raúl · La Entrada	25	L 12,450.00	44.0%	SALDO L 7,450.00 · vence 10 oct	PDF VER
#1498	6 sep	Repuestos Automotrices Dago · SPS	31	L 21,300.00	40.8%	PAGADA	PDF VER
TOTAL SEPTIEMBRE · 15 VENTAS (se muestran 6)			592	L 116,954.10	35.3%	cobrado 76% · saldo L 28,039	

ESTADO SIGUIENTE · DESPLEGABLE EN "CAMBIOS"

VER: CAMBIOS ▾ | ESTE MES | ESTE AÑO | AÑO ANTERIOR

Abonos, factura inicial, edición y respaldo semanal

Factura	Fecha	Cliente	Código	Descripción	Unidades
#1502	22 sep	Inversiones Piky	47201-60460	Hilux 89-95 · Land Cruiser · Runner · 15/16"	1
#1494	2 sep	Auto Repuestos Navarro	31410-35090	Hilux 79-88 · Runner 84-88	1
#1471	18 ago	Taller Santos	47550-26090	Hilux 79-88 · Hiace 89 · 7/8" · abajo	2
#1433	26 jun	Distribuidora Eliana	47201-04150	Tacoma 05-15	1
TOTAL 2026 · 9 CAMBIOS (se muestran 4)					12 u

ESTADO SIGUIENTE · BUSCADOR: ESCRIBIR "NAV" Y ELEGIR AUTO REPUESTOS NAVARRO

AUTO REPUESTOS NAVARRO
Catacamas, Olancho · 9832-1315 · crédito 60 días

VENCIDO L 0.00 · DEBE EN TOTAL L 16,620.00 · + ABONO

Factura	Fecha	Vence	Monto inicial	Abonado	Pendiente	
#1488	20 ago	19 oct · faltan 2 días	L 18,240.00	L 11,460.00	L 6,780.00	ver · editar ›
#1503	24 sep	23 nov · faltan 37 días	L 9,840.00	L 0.00	L 9,840.00	ver · editar ›

ESTADO SIGUIENTE · "+ ABONO"

FACTURA: #1488 · L 6,780.00 | FECHA: 17 oct 2026 | MONTO: L 6,780.00 | Nº RECIBO: R-0862 | MEDIO DE PAGO: EFECTIVO · TRANSFERENCIA · CHEQUE · DEVOLUCIÓN | GUARDAR ABONO

ESTADO SIGUIENTE · "+ FACTURA INICIAL" · SOLO EN ARRANQUE

Nº FACTURA: 1478 | FECHA: 12 ago 2026 | CLIENTE: Inversiones Piky | MONTO INICIAL: L 14,820.00 | ABONADO: L 8,000.00 | SALDO: L 6,820.00 | GUARDAR

ESTADO SIGUIENTE · "CORREGIR SALDO" · SOLO FACTURAS DE CARGA INICIAL

FACTURA: #1478 · carga inicial | SALDO: L 6,820.00 → L 7,320.00 | MOTIVO (OBLIGATORIO): abono mal restado al cargar | CORREGIR

ESTADO SIGUIENTE · EDITAR FACTURA #1488 POR DEVOLUCIÓN

FACTURA #1488 · 20 AGO 2026 · NAVARRO

se recalcula en agosto, el mes en que se emitió

Código	Descripción	Cant. original	Cant. nueva	Precio	Total
47201-60460	Hilux 89-95 · Land Cruiser · Runner · 15/16"	16	14	L 720.00	L 10,080.00
31410-35090	Hilux 79-88 · Runner 84-88	16	16	L 420.00	L 6,720.00

Factura L 18,240.00 → L 16,800.00 · abonado L 11,460.00 · pendiente L 6,780.00 → L 5,340.00 | GUARDAR CAMBIO

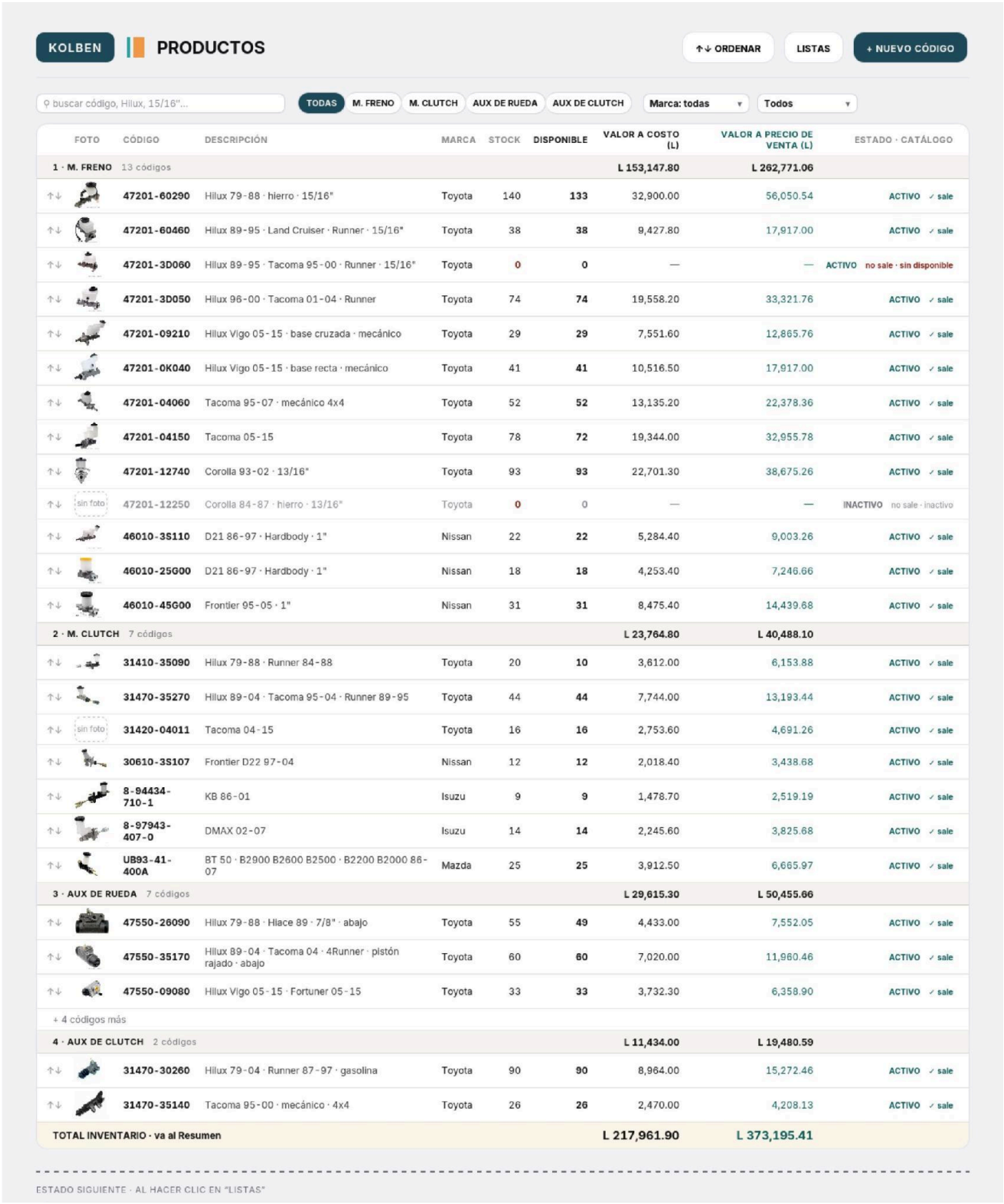
RESPALDO SEMANAL · PDF QUE LLEGA CADA LUNES AL CORREO

KOLBEN · CUENTAS POR COBRAR — al lunes 19 oct 2026

OLANCHO · CATACAMAS				
Auto Repuestos Navarro	#1488	vence 19 oct	L 6,780.00	
	#1503	vence 23 nov	L 9,840.00	L 16,620.00
Distribuidora Eliana	#1500	vencida 4 d	L 9,860.00	L 9,860.00
COPÁN · LA ENTRADA				
Inversiones Piky	#1478	vencida 38 d	L 6,820.00	
	#1496	vencida 16 d	L 5,340.50	L 12,160.50
... todas las zonas y clientes, completo, sin cortes				
TOTAL POR COBRAR · 12 facturas · vencido L 91,164.00				L 115,384.00

A.10 Pantalla 8 · Productos · Etapa 2

Productos y listas



KOLBEN | PRODUCTOS

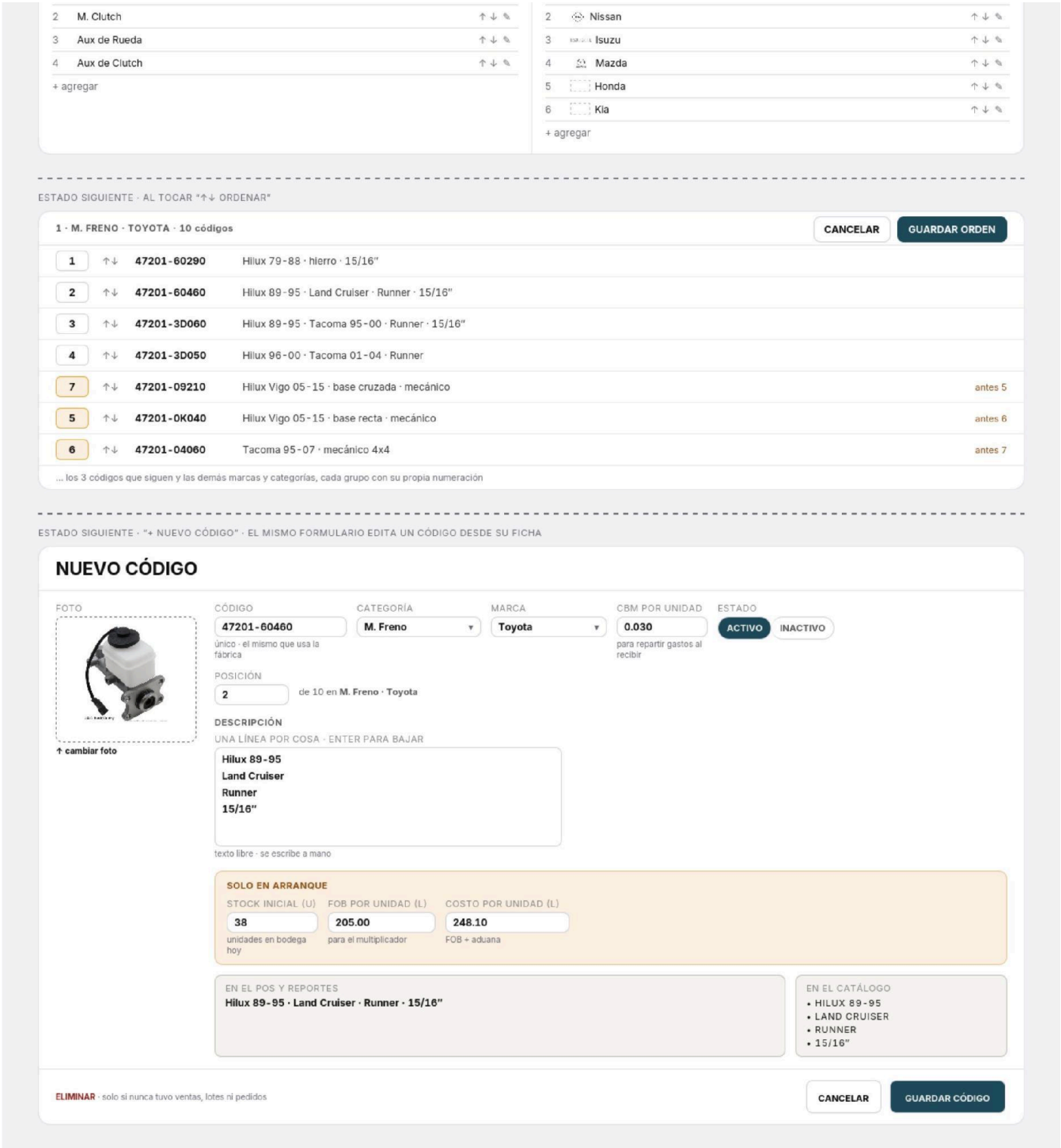
↑↓ ORDENAR | LISTAS | + NUEVO CÓDIGO

⚲ buscar código, Hilux, 15/16"... | TODAS | M. FRENO | M. CLUTCH | AUX DE RUEDA | AUX DE CLUTCH | Marca: todas | Todos

	FOTO	CÓDIGO	DESCRIPCIÓN	MARCA	STOCK	DISPONIBLE	VALOR A COSTO (L)	VALOR A PRECIO DE VENTA (L)	ESTADO · CATÁLOGO
1 · M. FRENO	13 códigos						**L 153,147.80**	**L 262,771.06**	
↑↓		**47201-60290**	Hilux 79-88 · hierro · 15/16"	Toyota	140	**133**	32,900.00	56,050.54	ACTIVO ✓ sale
↑↓		**47201-60460**	Hilux 89-95 · Land Cruiser · Runner · 15/16"	Toyota	38	**38**	9,427.80	17,917.00	ACTIVO ✓ sale
↑↓		**47201-3D060**	Hilux 89-95 · Tacoma 95-00 · Runner · 15/16"	Toyota	**0**	**0**	—	—	ACTIVO no sale · sin disponible
↑↓		**47201-3D050**	Hilux 96-00 · Tacoma 01-04 · Runner	Toyota	74	**74**	19,558.20	33,321.76	ACTIVO ✓ sale
↑↓		**47201-09210**	Hilux Vigo 05-15 · base cruzada · mecánico	Toyota	29	**29**	7,551.60	12,865.76	ACTIVO ✓ sale
↑↓		**47201-0K040**	Hilux Vigo 05-15 · base recta · mecánico	Toyota	41	**41**	10,516.50	17,917.00	ACTIVO ✓ sale
↑↓		**47201-04060**	Tacoma 95-07 · mecánico 4x4	Toyota	52	**52**	13,135.20	22,378.36	ACTIVO ✓ sale
↑↓		**47201-04150**	Tacoma 05-15	Toyota	78	**72**	19,344.00	32,955.78	ACTIVO ✓ sale
↑↓		**47201-12740**	Corolla 93-02 · 13/16"	Toyota	93	**93**	22,701.30	38,675.26	ACTIVO ✓ sale
↑↓	sin foto	47201-12250	Corolla 84-87 · hierro · 13/16"	Toyota	**0**	0	—	—	INACTIVO no sale · inactivo
↑↓		**46010-3S110**	D21 86-97 · Hardbody · 1"	Nissan	22	**22**	5,284.40	9,003.26	ACTIVO ✓ sale
↑↓		**46010-25G00**	D21 86-97 · Hardbody · 1"	Nissan	18	**18**	4,253.40	7,246.66	ACTIVO ✓ sale
↑↓		**46010-45G00**	Frontier 95-05 · 1"	Nissan	31	**31**	8,475.40	14,439.68	ACTIVO ✓ sale
2 · M. CLUTCH	7 códigos						**L 23,764.80**	**L 40,488.10**	
↑↓		**31410-35090**	Hilux 79-88 · Runner 84-88	Toyota	20	**10**	3,612.00	6,153.88	ACTIVO ✓ sale
↑↓		**31470-35270**	Hilux 89-04 · Tacoma 95-04 · Runner 89-95	Toyota	44	**44**	7,744.00	13,193.44	ACTIVO ✓ sale
↑↓	sin foto	**31420-04011**	Tacoma 04-15	Toyota	16	**16**	2,753.60	4,691.26	ACTIVO ✓ sale
↑↓		**30610-3S107**	Frontier D22 97-04	Nissan	12	**12**	2,018.40	3,438.68	ACTIVO ✓ sale
↑↓		**8-94434-710-1**	KB 86-01	Isuzu	9	**9**	1,478.70	2,519.19	ACTIVO ✓ sale
↑↓		**8-97943-407-0**	DMAX 02-07	Isuzu	14	**14**	2,245.60	3,825.68	ACTIVO ✓ sale
↑↓		**UB93-41-400A**	BT 50 · B2900 B2600 B2500 · B2200 B2000 86-07	Mazda	25	**25**	3,912.50	6,665.97	ACTIVO ✓ sale
3 · AUX DE RUEDA	7 códigos						**L 29,615.30**	**L 50,455.66**	
↑↓		**47550-26090**	Hilux 79-88 · Hiace 89 · 7/8" · abajo	Toyota	55	**49**	4,433.00	7,552.05	ACTIVO ✓ sale
↑↓		**47550-35170**	Hilux 89-04 · Tacoma 04 · 4Runner · pistón rajado · abajo	Toyota	60	**60**	7,020.00	11,960.46	ACTIVO ✓ sale
↑↓		**47550-09080**	Hilux Vigo 05-15 · Fortuner 05-15	Toyota	33	**33**	3,732.30	6,358.90	ACTIVO ✓ sale
+ 4 códigos más									
4 · AUX DE CLUTCH	2 códigos						**L 11,434.00**	**L 19,480.59**	
↑↓		**31470-30260**	Hilux 79-04 · Runner 87-97 · gasolina	Toyota	90	**90**	8,964.00	15,272.46	ACTIVO ✓ sale
↑↓		**31470-35140**	Tacoma 95-00 · mecánico · 4x4	Toyota	26	**26**	2,470.00	4,208.13	ACTIVO ✓ sale
TOTAL INVENTARIO · va al Resumen							**L 217,961.90**	**L 373,195.41**	

ESTADO SIGUIENTE · AL HACER CLIC EN "LISTAS"

Orden y nuevo código



A.11 Pantalla 9 · Ficha de código · Etapa 3

Ficha de código, lotes y ajustes

KOLBEN | CÓDIGO

← VOLVER A PRODUCTOS



47201-60460

✎ EDITAR

TOYOTA · M. FRENO · ACTIVO

Hilux 89-95 · Land Cruiser · Runner · 15/16"

CBM por unidad 0.030 · sale en el catálogo

STOCK	APARTADO	DISPONIBLE	VALOR DEL STOCK · A COSTO	VALOR A PRECIO DE VENTA	COSTO DE LO QUE SALE AHORA
38	0	38	L 9,427.80	L 17,917.00	L 248.10 por unidad · lote pedido 24

LOTES · FIFO · SALE PRIMERO EL MÁS VIEJO — solo lectura · se crean al recibir un pedido

PEDIDO	RECIBIDO	CANTIDAD	QUEDAN	FOB/U (L)	ADUANA/U (L)	COSTO/U (L)
24	18 may 2026	143	**38**	205.00	43.10	**248.10**
23	20 nov 2025	120	0	196.40	41.80	238.20

2024 y antes · 4 lotes agotados · ver ›

AJUSTES DE STOCK — ± AJUSTAR STOCK

FECHA	UNIDADES	MOTIVO	STOCK
20 may 2026	−2	faltante pedido 24	145 → 143
12 ene 2026	+1	conteo físico	54 → 55

ESTADO SIGUIENTE · "AJUSTAR STOCK" · DOS PASOS

PASO 1 · QUÉ CAMBIA

TIPO: − SALE | + ENTRA — UNIDADES: 2 — MOTIVO (OBLIGATORIO): pieza dañada en bodega

Stock **38 → 36** — CONTINUAR

PASO 2 · CONFIRMAR

¿Sacar 2 unidades de 47201-60460?

Stock 38 → 36 · motivo: pieza dañada en bodega

No se puede deshacer; si te equivocas, se corrige con otro ajuste.

CANCELAR — SÍ, AJUSTAR

ESTADO SIGUIENTE · CÓDIGO CARGADO EN EL ARRANQUE · CORREGIR EL COSTO DEL LOTE "INICIAL"

LOTES · FIFO

PEDIDO	RECIBIDO	CANTIDAD	QUEDAN	FOB/U (L)	COSTO/U (L)
INICIAL	4 ene 2027 · arranque	38	22	205.00 ✎	**248.10** ✎

CONFIRMAR CAMBIO DE COSTO

Lote Inicial · L 248.10 → L 262.00

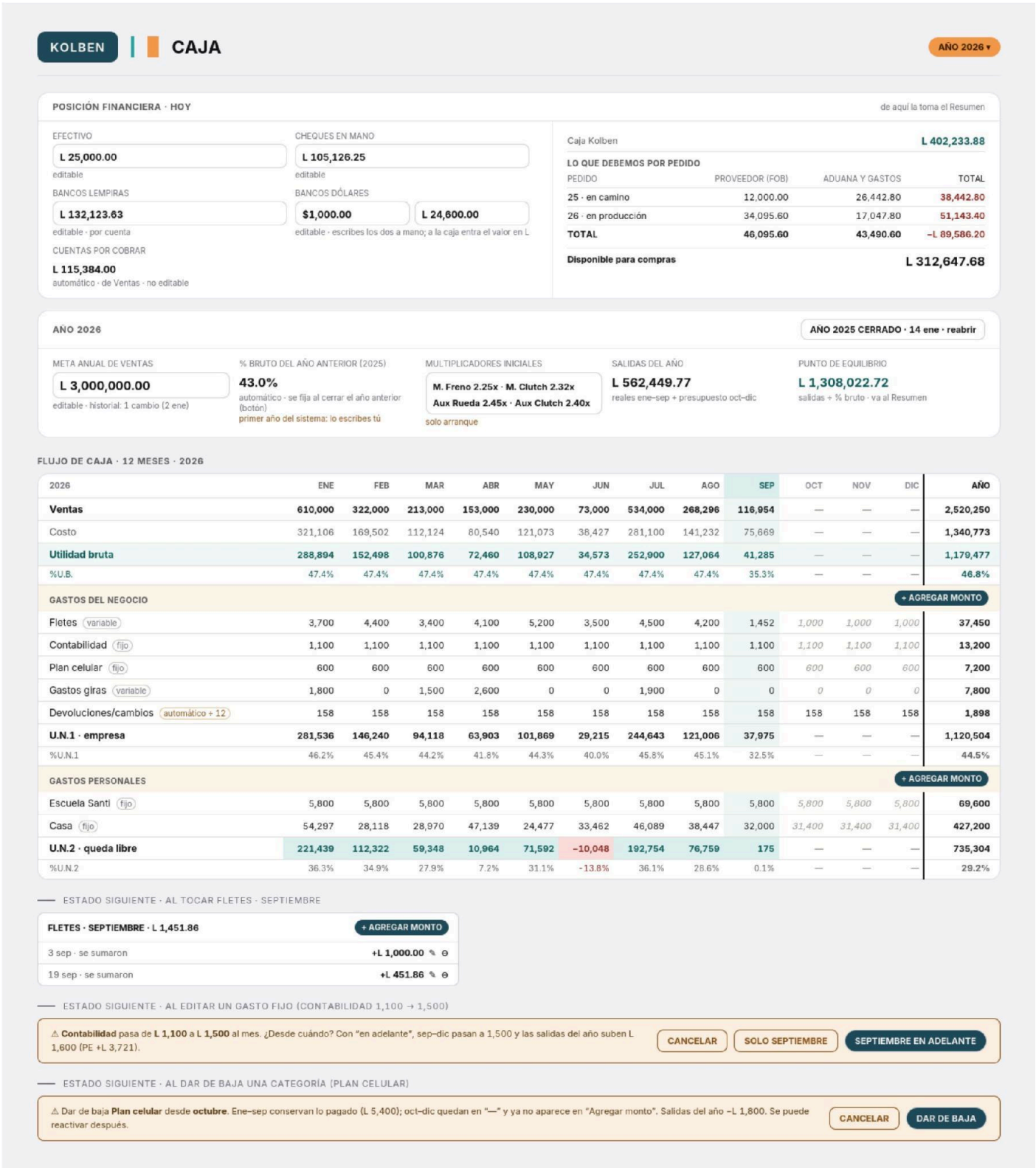
Motivo (obligatorio): **costo mal escrito en la carga**

Las 16 unidades ya vendidas de este lote recalculan su utilidad en el mes de cada venta. Queda en el historial del código.

CANCELAR — SÍ, CAMBIAR

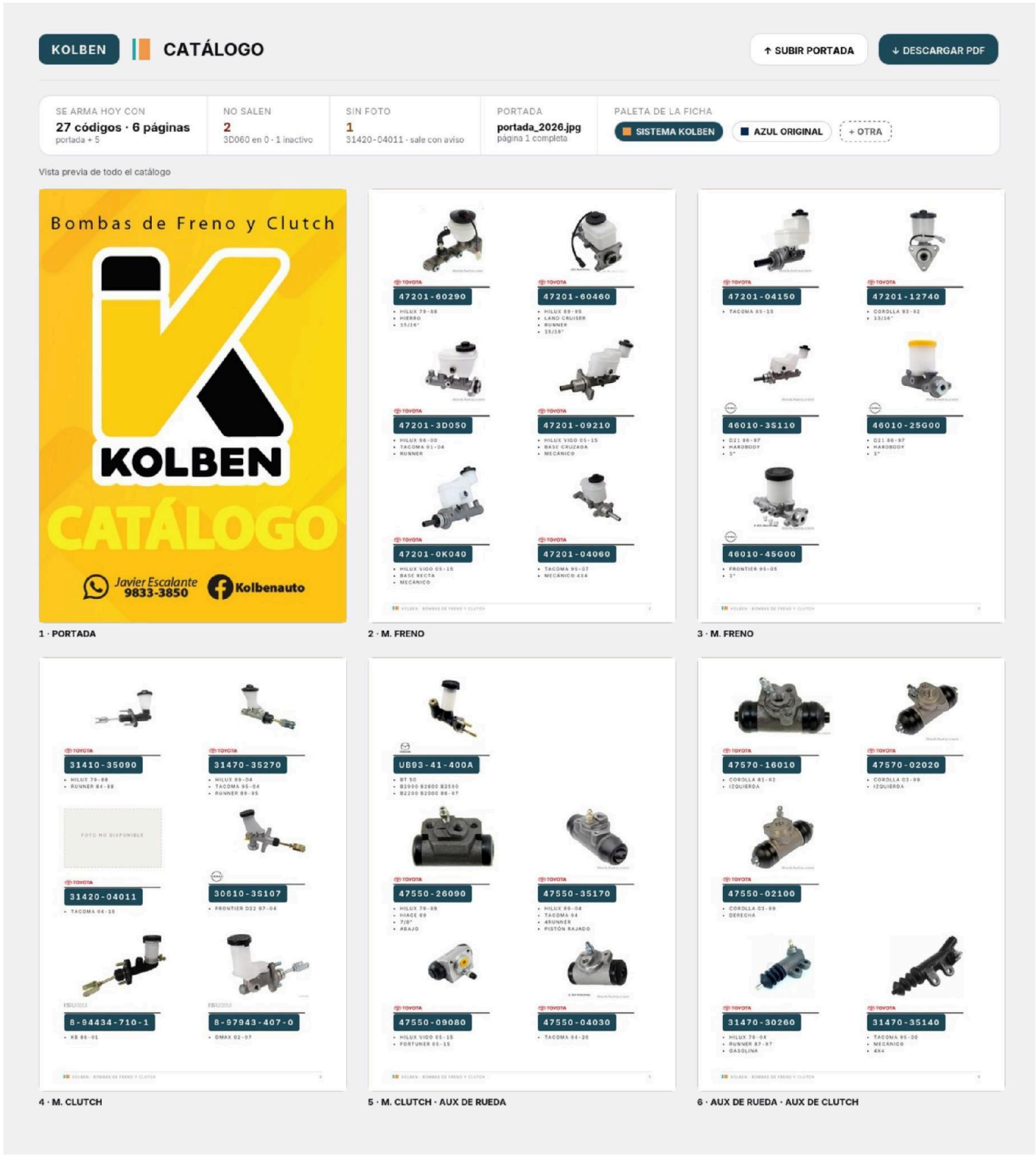
A.12 Pantalla 10 · Caja · Etapa 11

Caja y flujo de 12 meses



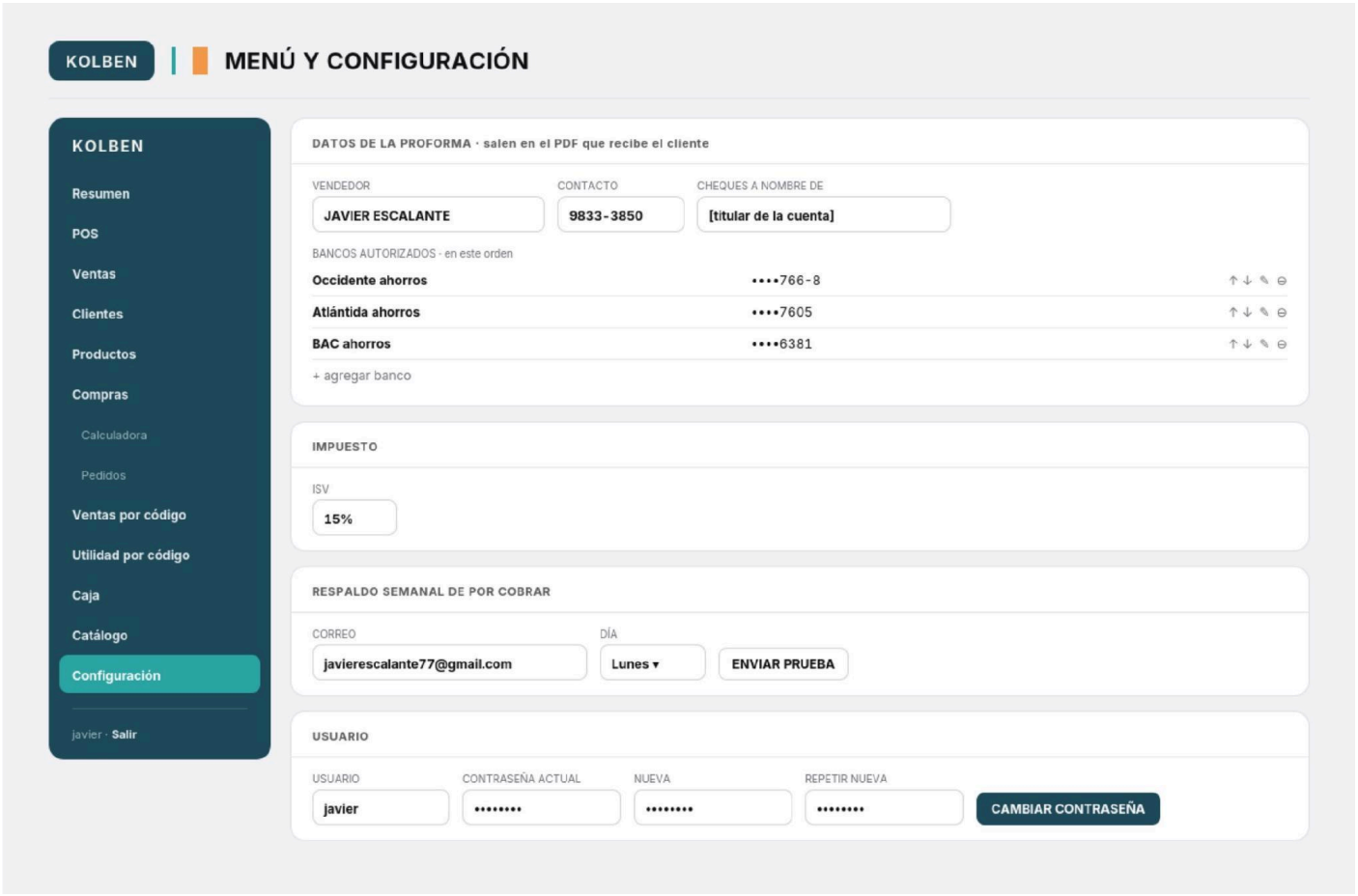
A.13 Pantalla 11 · Catálogo · Etapa 13

Catálogo PDF



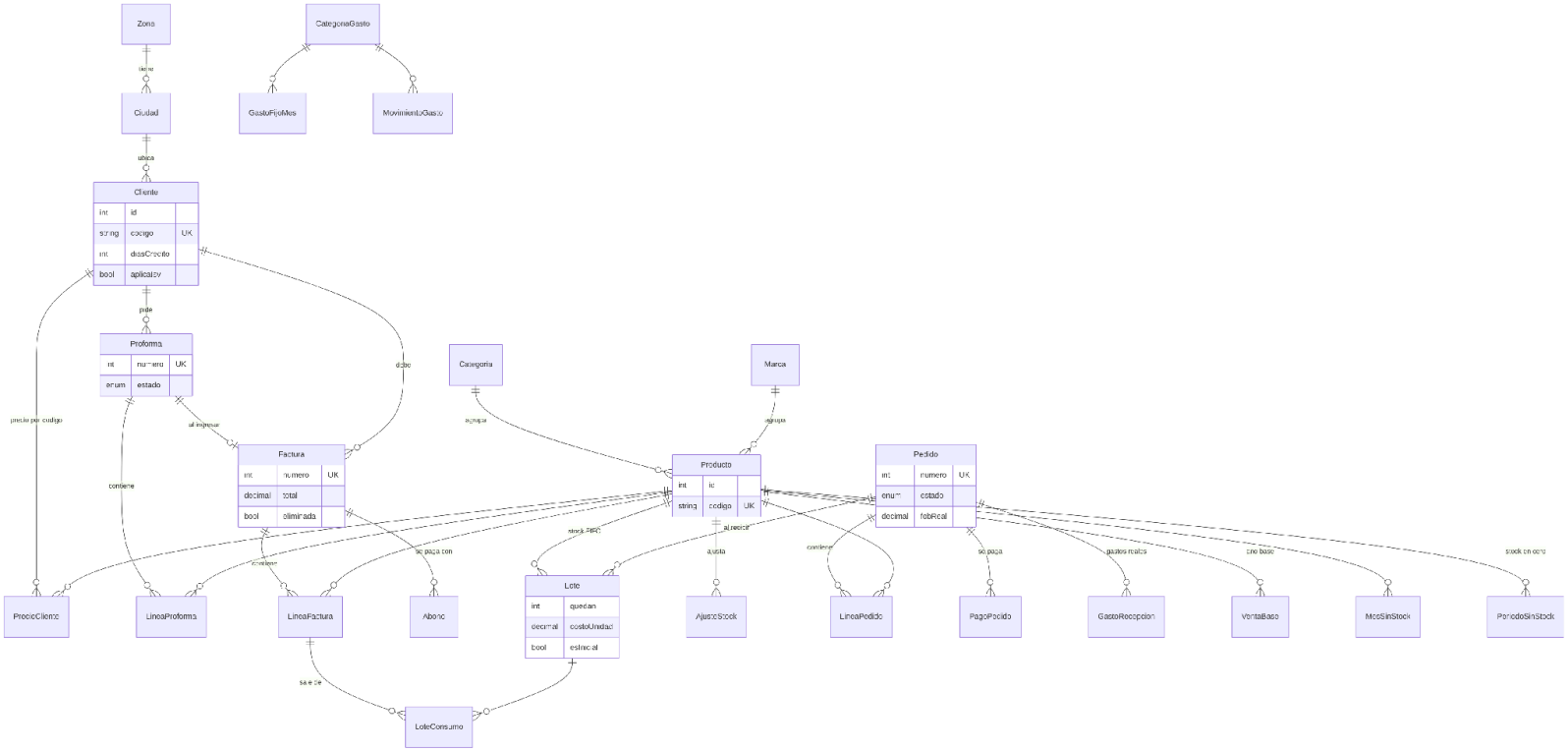
A.14 Pantalla 12 · Menú y Configuración · Etapa 1

Menú y Configuración



Anexo B. Diagrama de relaciones del modelo de datos

Tablas principales y sus relaciones. No muestra todas las columnas (están en el esquema Prisma de la sección 9) ni la tabla `Historial`, que se conecta con todas. Cada línea indica “uno a muchos”.



Bloques del modelo

- **Catálogo y clientes:** `Producto` (agrupado por `Categoria` y `Marca`), `Cliente` (en una `Ciudad`, dentro de una `Zona`) y `PrecioCliente`.
- **Ventas:** `Proforma` y sus líneas; al ingresar el pedido se crea la `Factura` con sus `LineaFactura` y `Abono`.
- **Inventario FIFO:** el stock es la suma de `quedan` de los `Lote`; `LoteConsumo` guarda de qué lote salió cada pieza; `AjusteStock` guarda las correcciones; `PeriodoSinStock` registra cuándo un código estuvo en cero.
- **Compras:** `Pedido` con líneas, pagos y gastos de recepción; al recibirlo crea los `Lote` con el costo real.
- **Caja y reportes:** categorías y movimientos de gasto, saldos, año y mes contable, `VentaBase`, `MesSinStock` e `Historial`.